Deep Learning

Yu Sheng

Center for Statistical Science

What enabled deep learning?

- Data

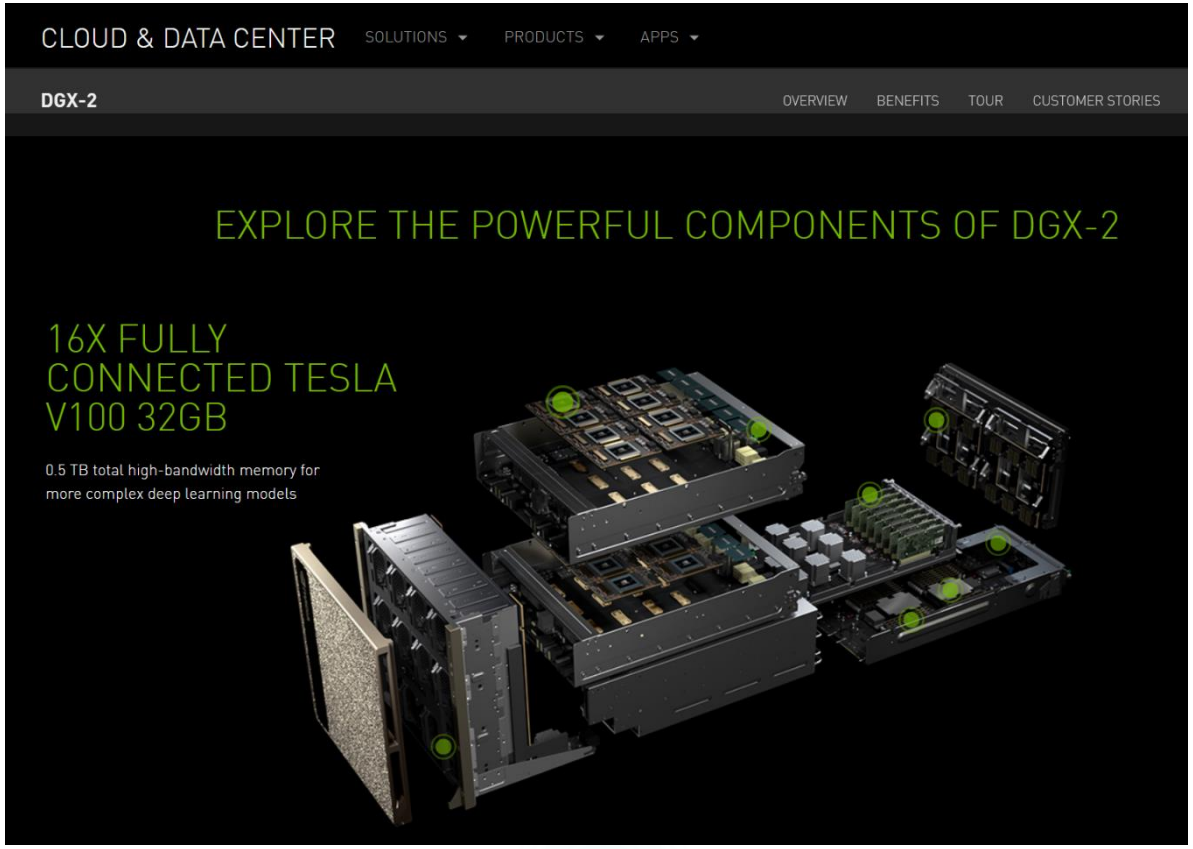


What enabled deep learning?

- New (relatively) affordable hardware

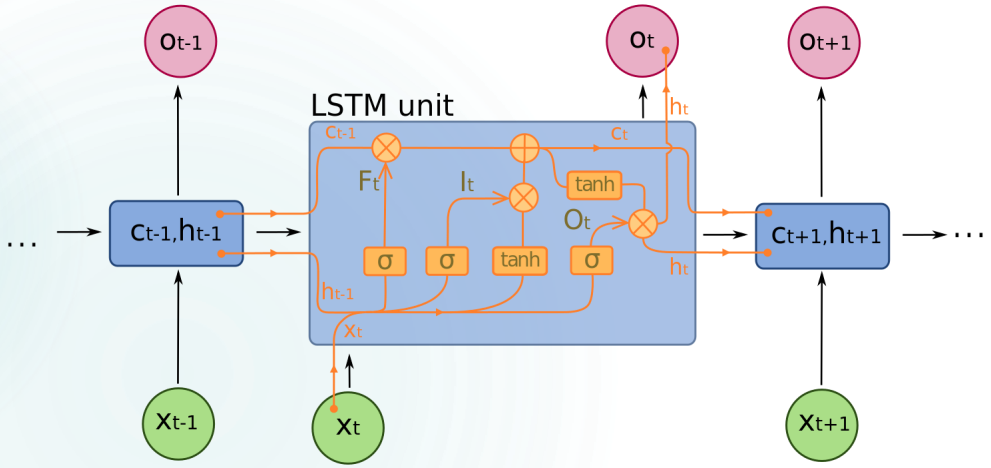


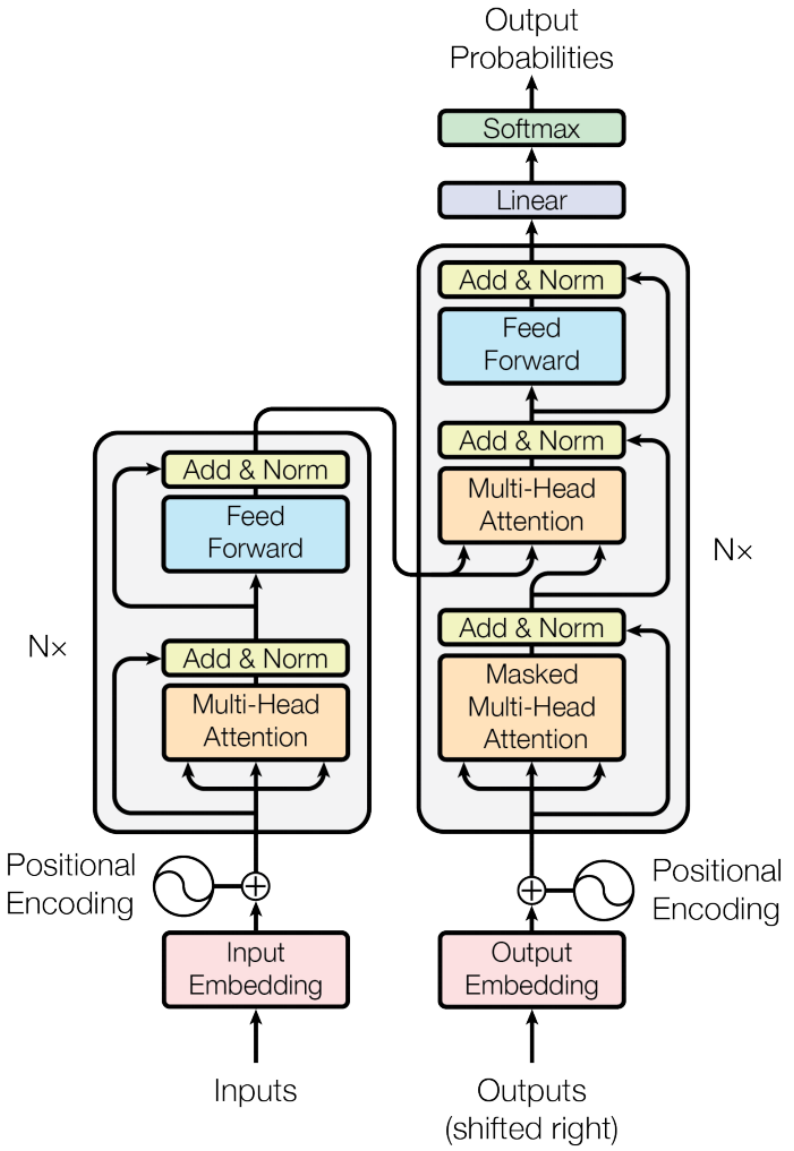
or



What enabled deep learning?

- New architectures

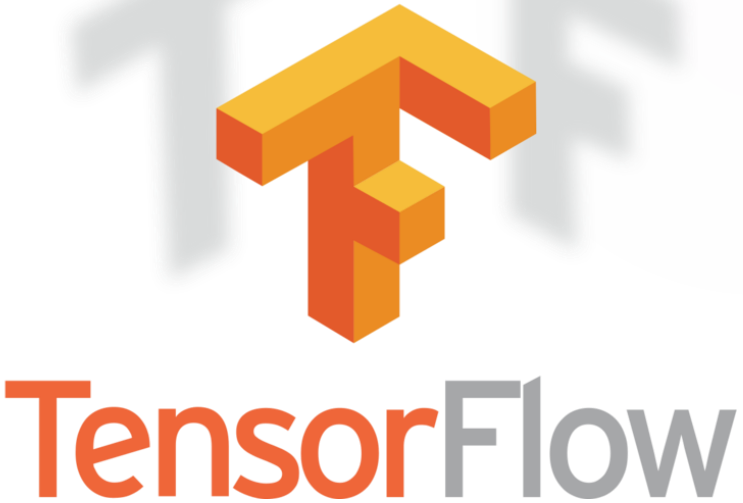




What enabled deep learning?

- Programming platforms
  - Deep learning as automatic differentiation





Outline

- Convolutional neural networks
  - Implemented as early as 1998 (LeCunn).
  - Became popular with the use of GPU, around 2005.
- Recurrent neural networks
  - Long short term memory (Sepp Hochreiter & Jürgen Schmidhuber, 1997).
  - Became popular since around 2007.
- Road to large language models

Convolutional Neural Networks

Credit

- The content of this section is largely based on:

  http://colah.github.io/posts/2014-07-Conv-Nets-Modular/

  http://cs231n.github.io/convolutional-networks/

  http://cs231n.github.io/understanding-cnn/

  http://googleresearch.blogspot.com/2015/06/inceptionism-going-deeper-into-neural.html

ConvNet Motivations

- Consider classifying images of size 200x200x3.
- A single fully connected neuron would require 120,000 weights.
- We would almost certainly want to have several such neurons, so the parameters would add up quickly!
- Clearly, this full connectivity is wasteful and the huge number of parameters would quickly lead to overfitting.

清华大学统计学研究中心

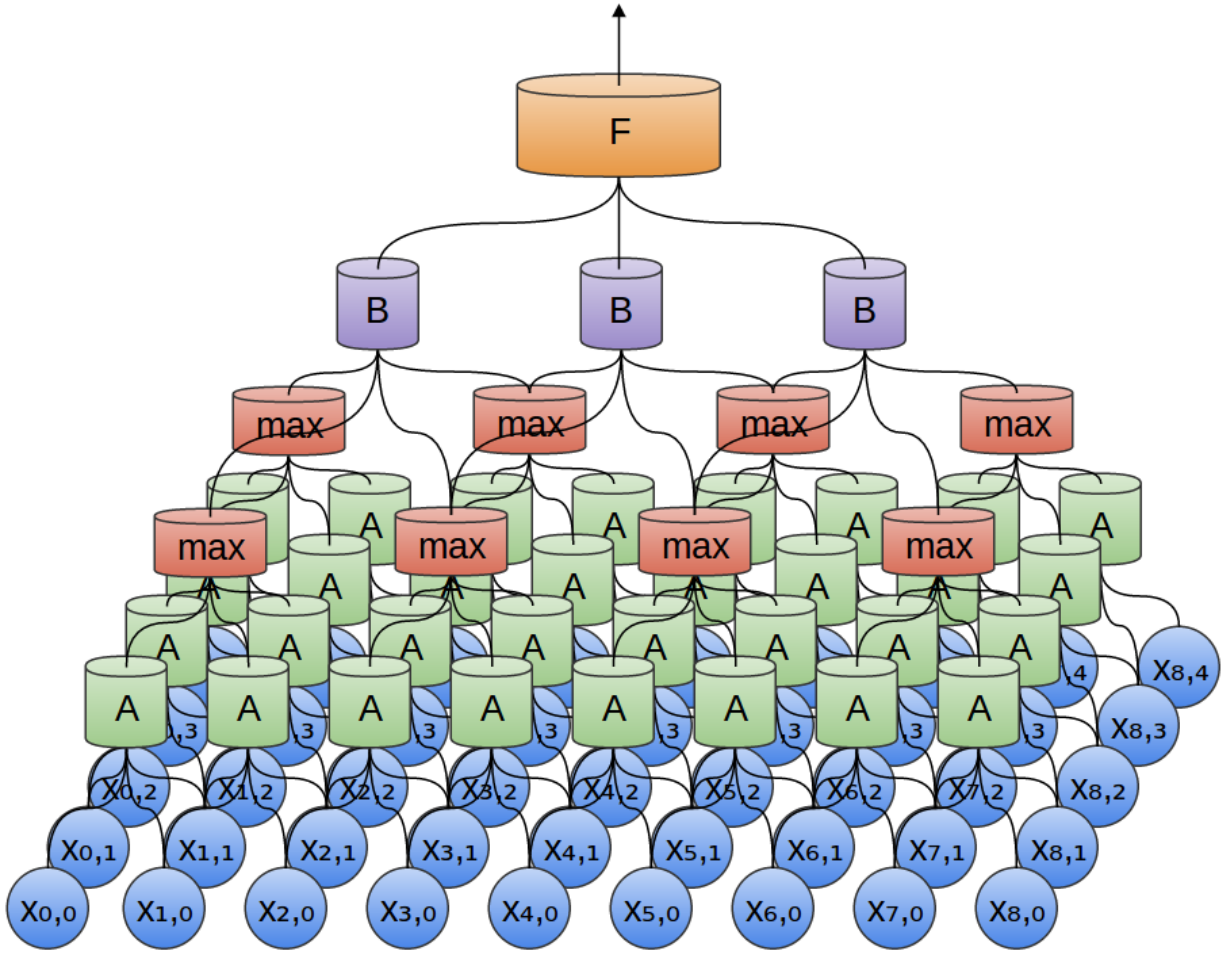
ConvNet Motivations

- A ConvNet has the following distinct features
  - Local Connectivity
  - Parameter Sharing
  - Max-Pooling

清华大学统计学研究中心

ConvNet Motivations

- So a ConvNet may eventually look like



Starting from the simplest

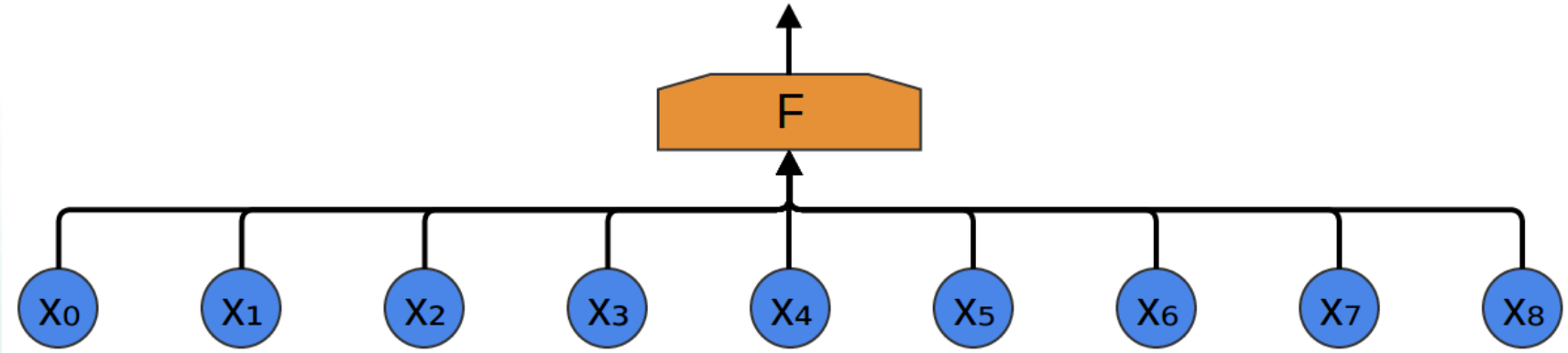
- Suppose we get an audio sample or a word sequence, which is 1-dimensional.



- In reality, each x_i can be a vector.

Starting from the simplest

- In our conventional neural network, we would connect it to a *Fully Connected* layer.

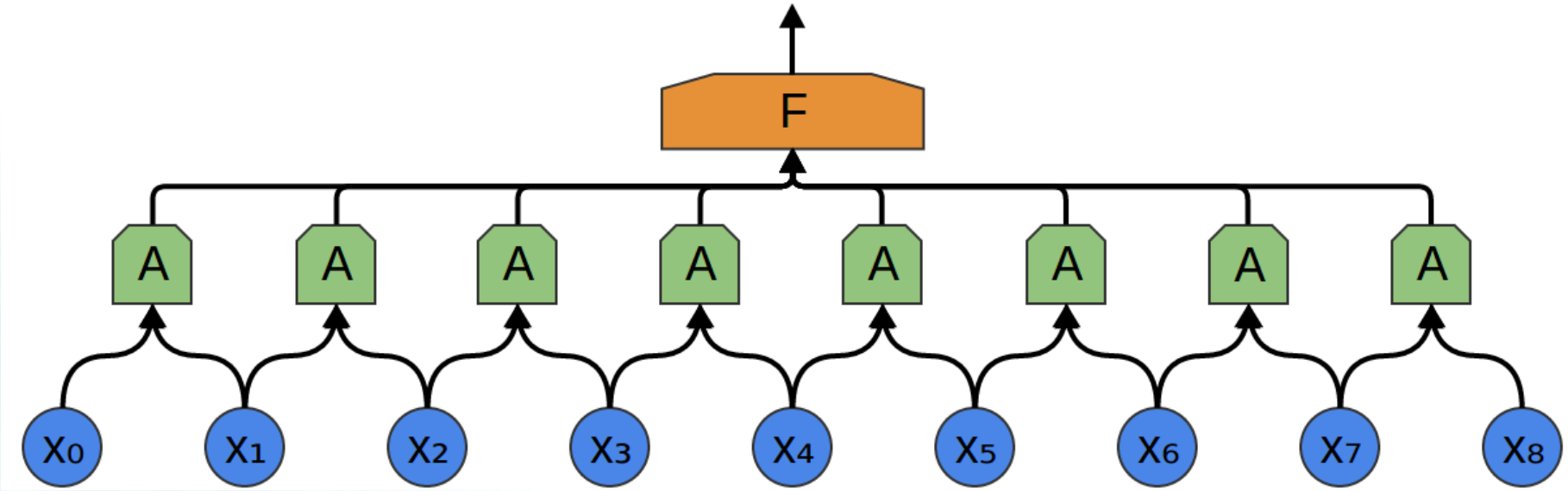


- But that's not a very good design for audio, sentences, or images.

清华大学统计学研究中心

Convolutional Layer

- Instead, we can create a group of neurons, *A*, that look at local properties. Then we feed this new layer to a fully connected layer.

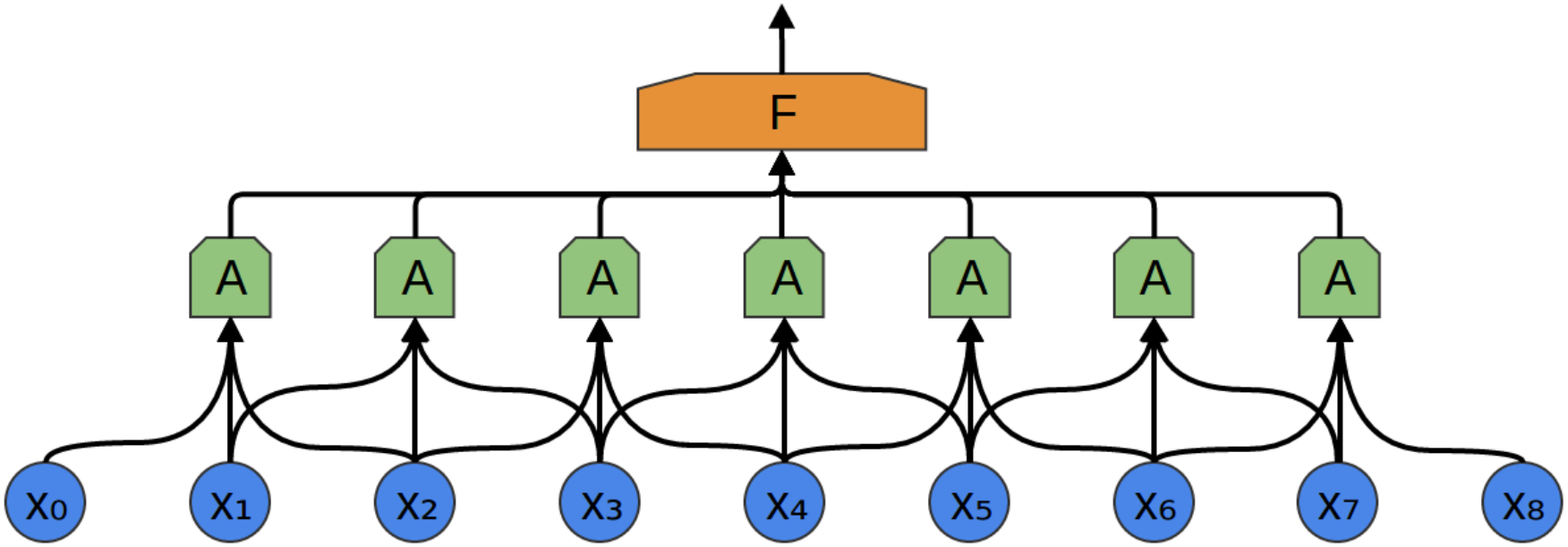


- The layer of *A* is called a *Convolutional Layer*.

清华大学统计学研究中心

Convolutional Layer

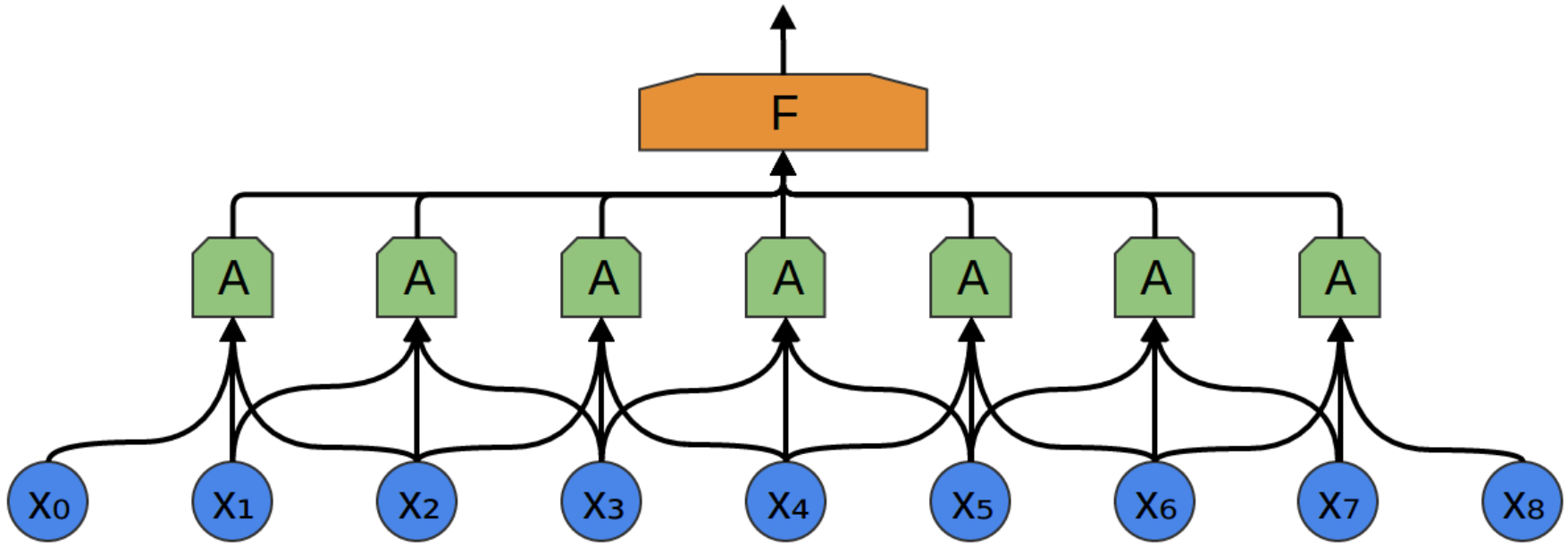
- In the previous example, *A* only looked at segments consisting of two points. Alternatively, it can look at more points, such as three.



清华大学统计学研究中心

Convolutional Layer

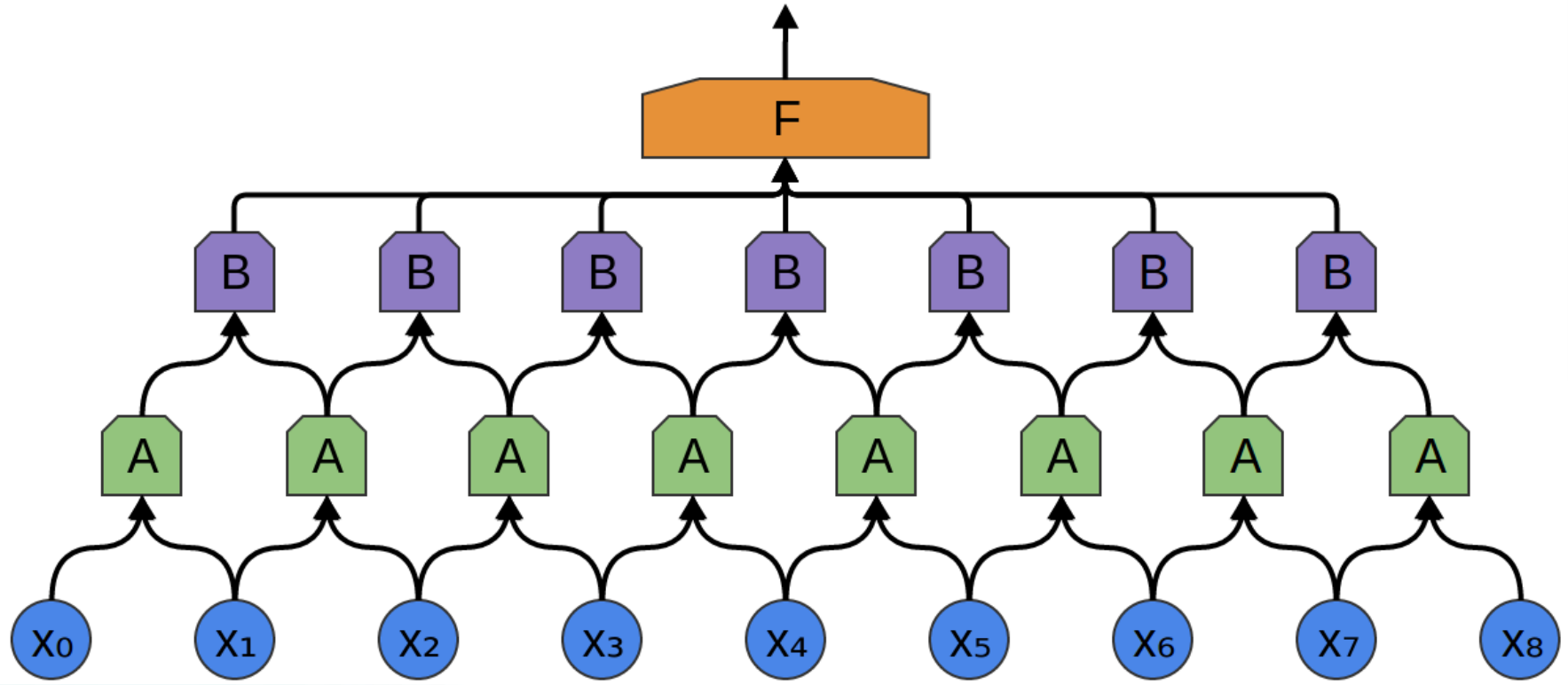
- Note that each *A* has the **same weights** to the neurons below. They are looking for the same features from different positions of the input, achieving **translational invariance**.



清华大学统计学研究中心

Convolutional Layer

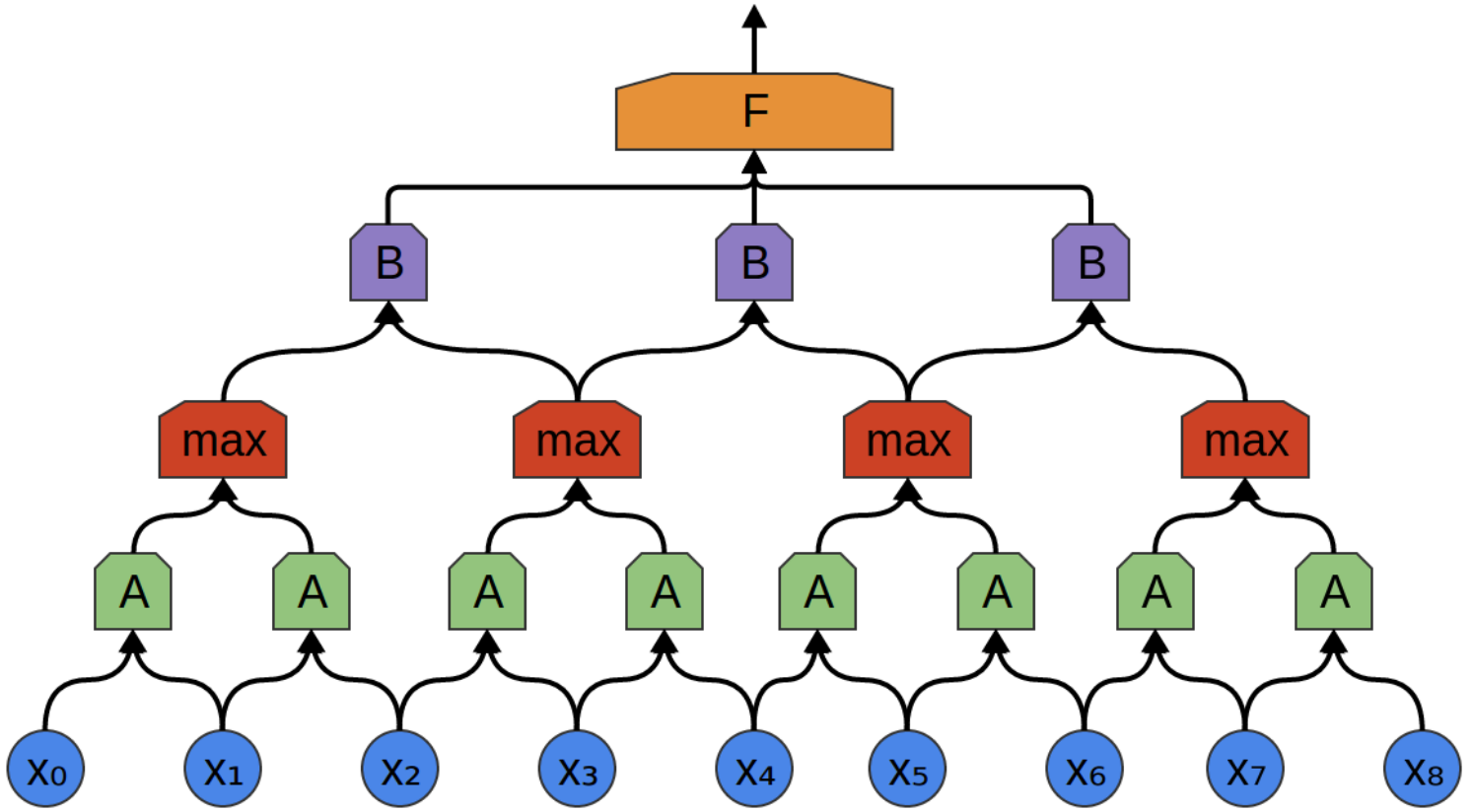
- We can feed the output of one convolutional layer into another. With each layer, the network can detect higher-level, more abstract features.



清华大学统计学研究中心

Max-Pooling

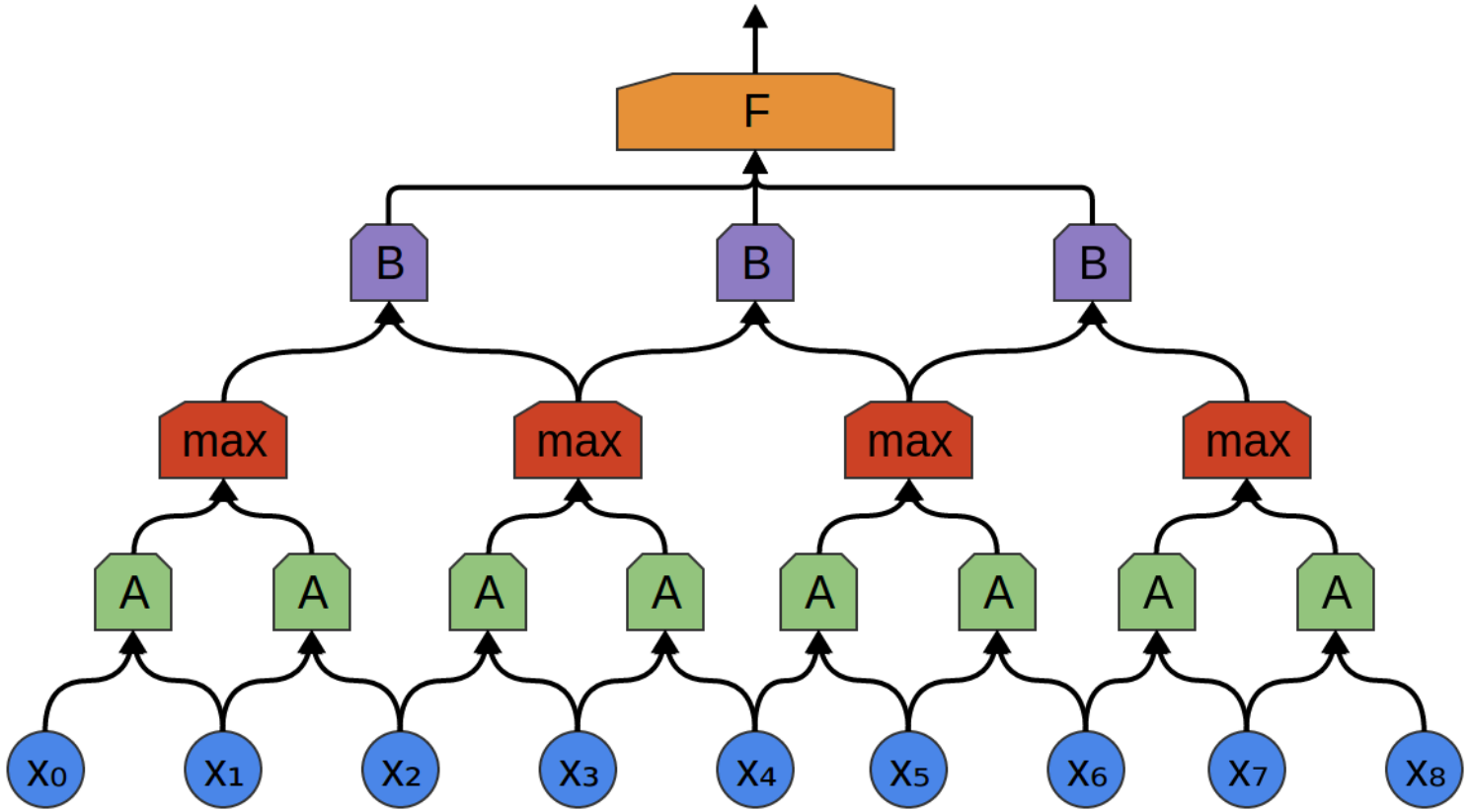
- Convolutional layers are often interweaved with pooling layers. In particular, there is a kind of layer called a max-pooling layer that is extremely popular.



清华大学统计学研究中心

Max-Pooling

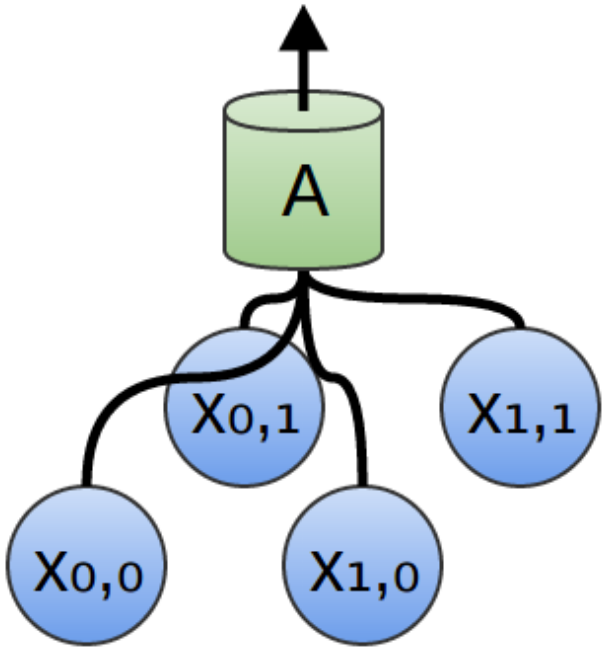
- Max-pooling layers kind of "zoom out". They allow later convolutional layers to work on larger sections of the data.



清华大学统计学研究中心

2D ConvNets

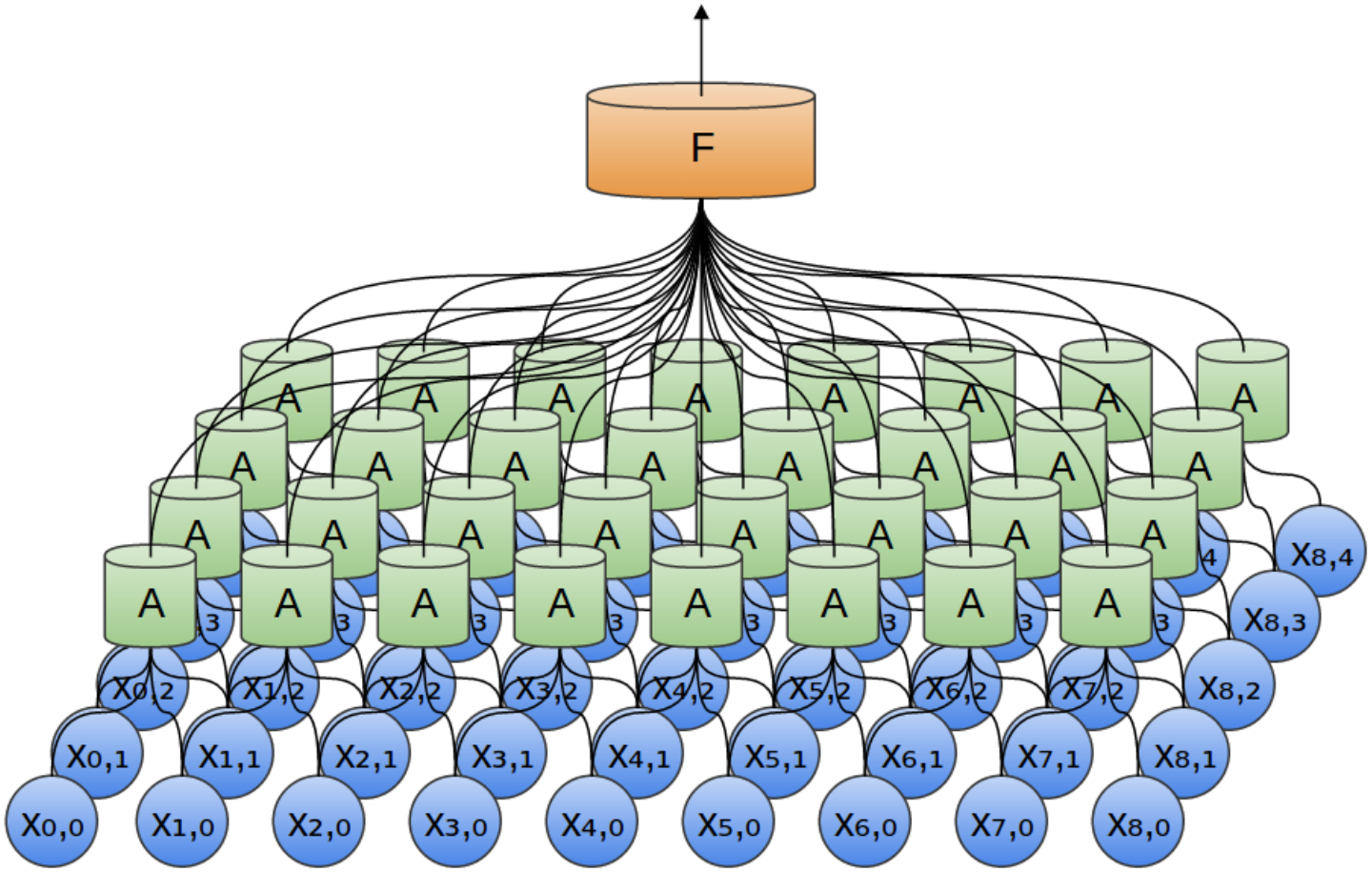
- In our previous examples, we've used 1-dimensional convolutional layers. However, the most famous successes of convolutional neural networks are applying 2D convolutional neural networks to recognizing images. And for this purpose, a convolutional unit *A* needs to connect to a 2D patch.



清华大学统计学研究中心

2D ConvNets

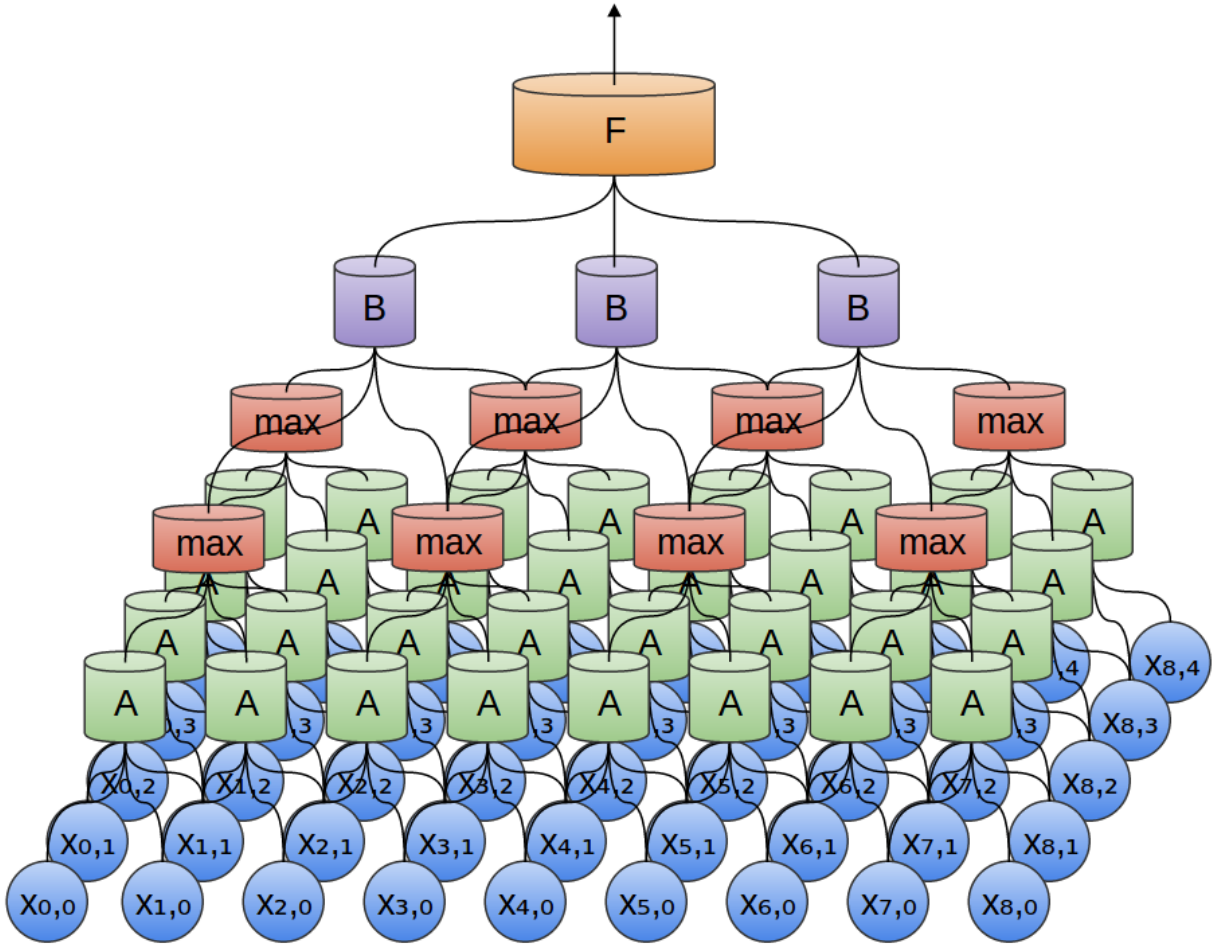
- So a single convolutional layer may look like this:



清华大学统计学研究中心

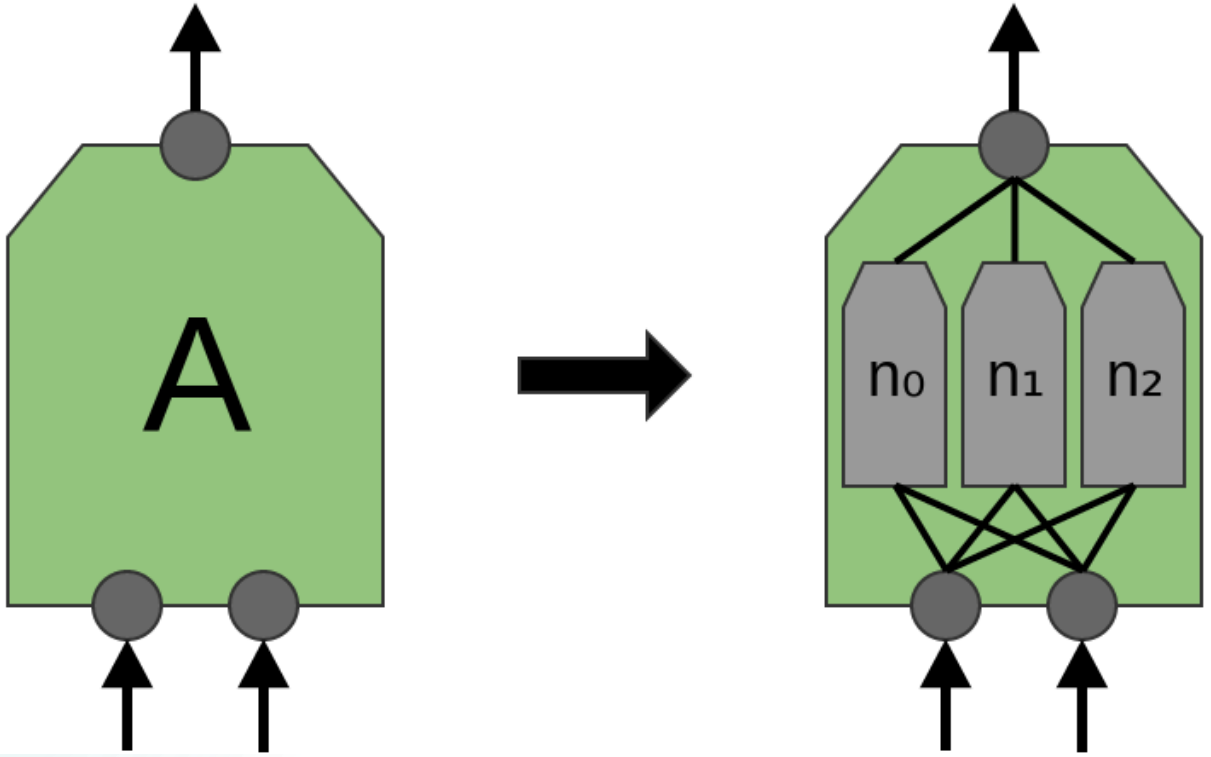
2D ConvNets

- And with more layers, the ConvNet may look like this:



2D ConvNets

- And recall that each convolutional unit is actually a set of feature:

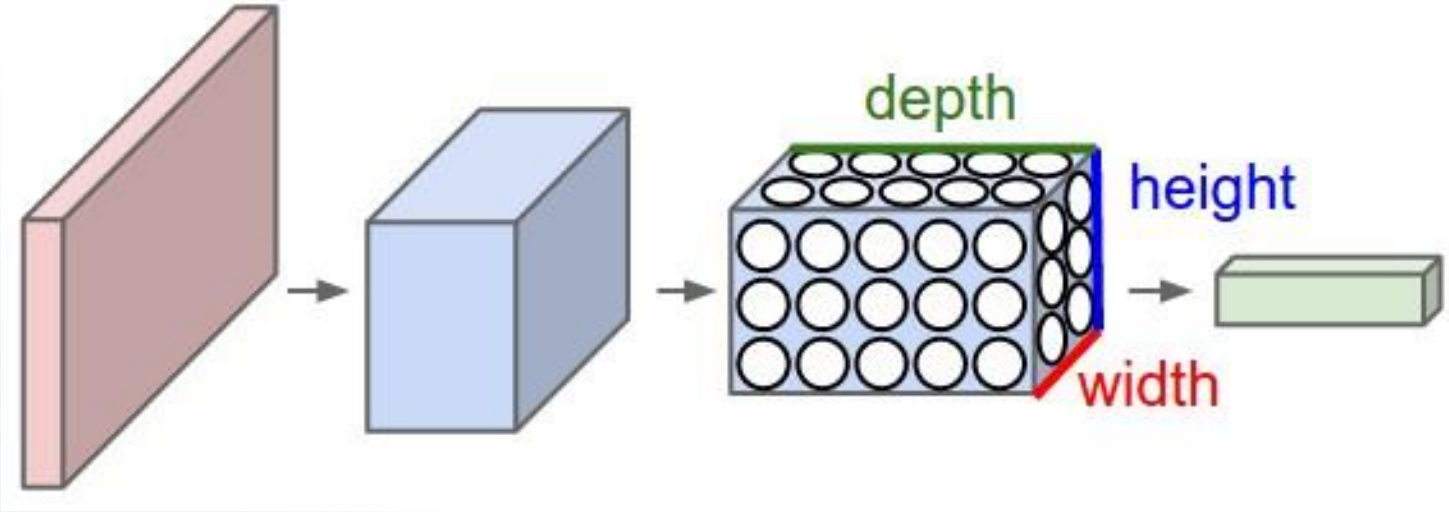


Apparently, we need a better framework for formulation and the actual coding.

清华大学统计学研究中心

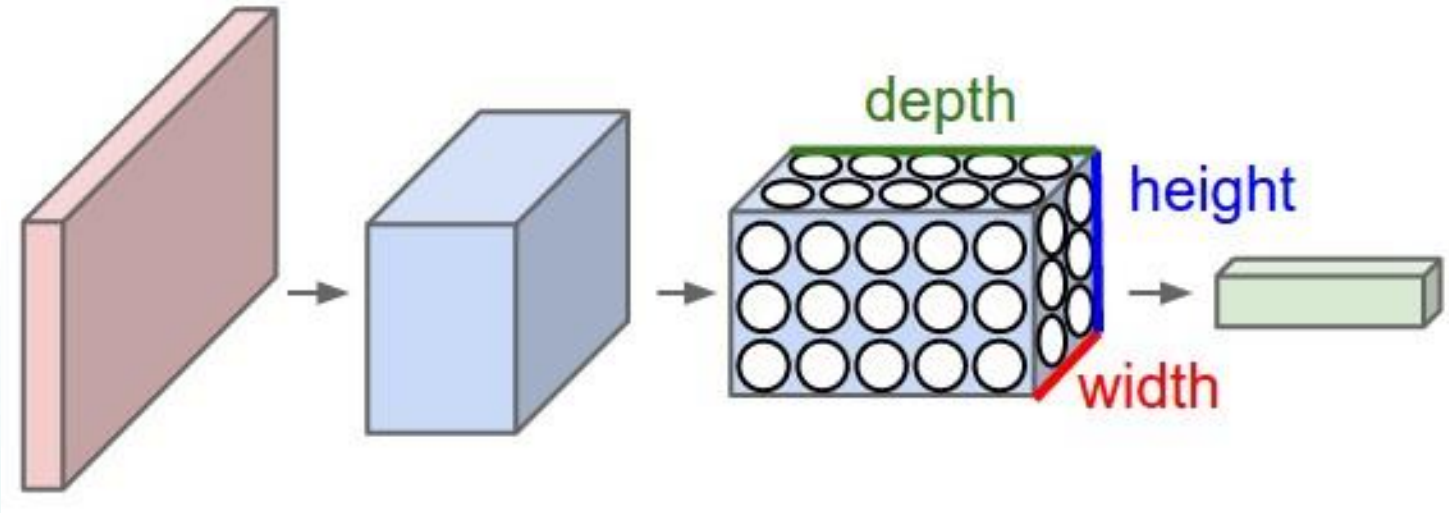
A Structure for the ConvNet Data

- A neat way to organize the data in each ConvNet layer is to use a 3D data volume.



清华大学统计学研究中心

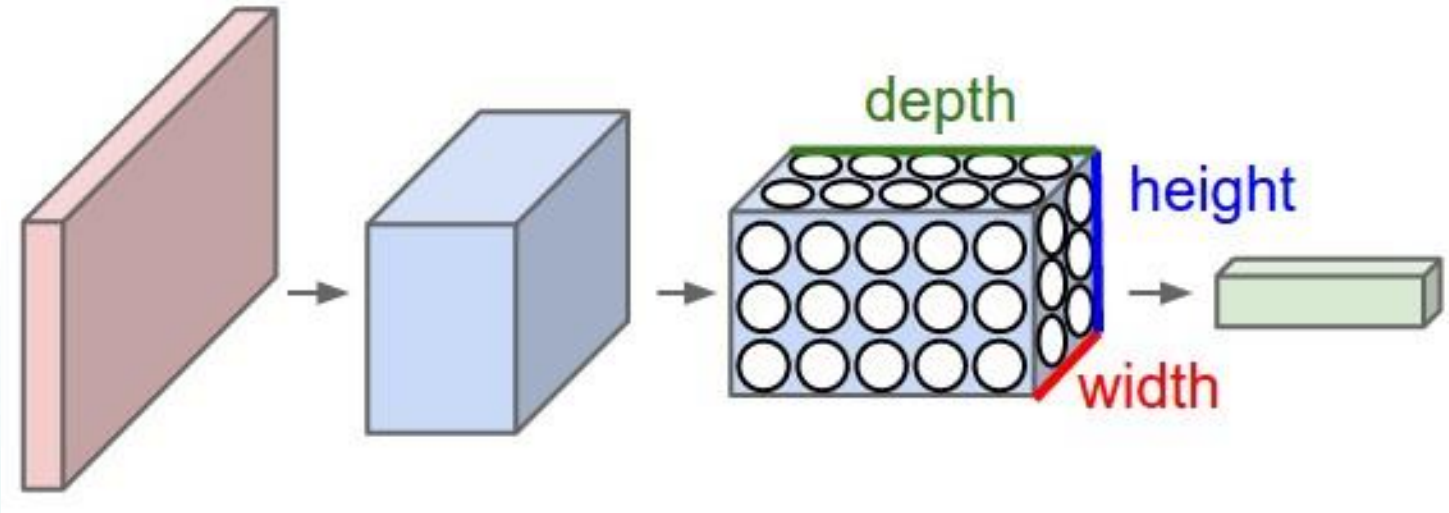
A Structure for the ConvNet Data



- The width and height come from the spatial information. The depth represents the number of features.
- For example, the raw input of an image could be 32x32x3, where 32x32 is the image size, and 3 is the RGB channels.

清华大学统计学研究中心

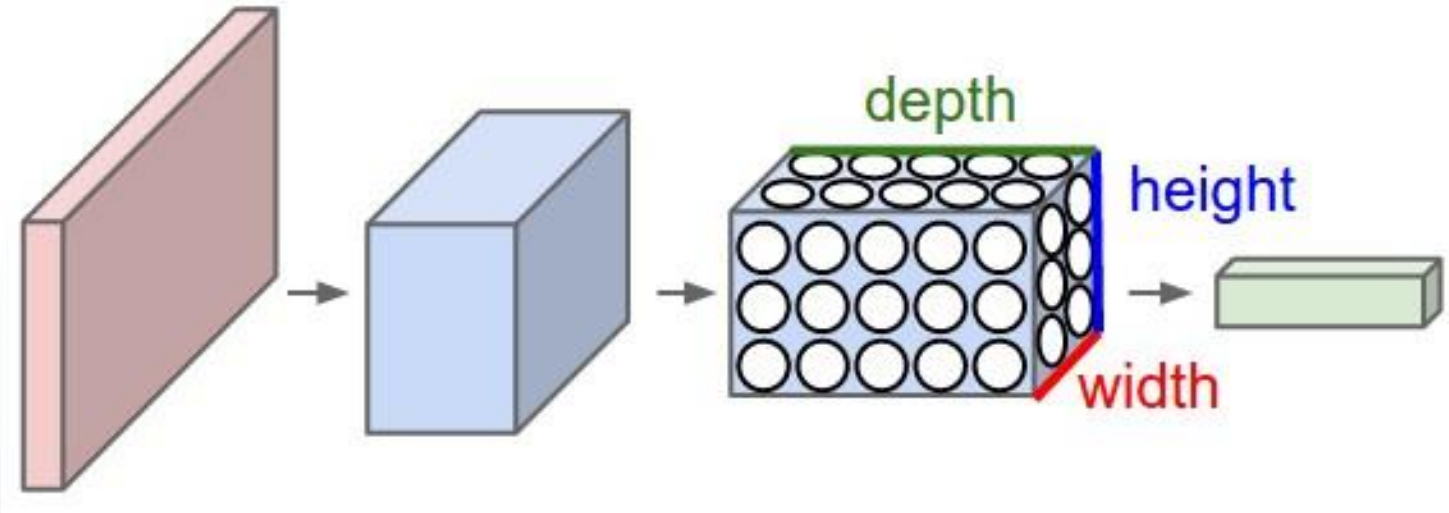
A Structure for the ConvNet Data



- In the middle layers, the width and height of the volume typically decrease due to convolution and pooling. The depth typically grows. Recall that *A* has multiple neurons in it, and the number of the neurons in *A* will be the depth.

清华大学统计学研究中心

A Structure for the ConvNet Data



- In the final few fully connected layers, the width and height typically have reduced to 1×1. The final output is typically 1×1×K, for a K-way classification.
- The layers we will use to build a ConvNet include INPUT, CONV, RELU, POOL, and FC. We see that this structure allows the layers to have a common API: the input is a 3D volume, and the output is again a 3D volume.

清华大学统计学研究中心

Convolutional Layer

- Local Connectivity: we will connect each neuron to only a local region of the input volume. The spatial extent of this connectivity is a hyperparameter called the **receptive field** of the neuron.

- The extent of the connectivity along the depth axis is always equal to the depth of the input volume. The connections are **local in space** (along width and height), but always **full along the entire depth** of the input volume.

清华大学统计学研究中心

Convolutional Layer

- *Example 1*. For example, suppose that the input volume has size [32×32×3], (e.g. an RGB CIFAR-10 image). If the receptive field is of size 5×5, then each neuron in the Conv Layer will have weights to a [5×5×3] region in the input volume, for a total of 5×5×3 = 75 weights. Notice that the extent of the connectivity along the depth axis must be 3, since this is the depth of the input volume.

- *Example 2*. Suppose an input volume had size [16×16×20]. Then using an example receptive field size of 3×3, every neuron in the Conv Layer would now have a total of 3×3×20 = 180 connections to the input volume. Notice that, again, the connectivity is local in space (e.g. 3×3), but full along the input depth (20).

清华大学统计学研究中心

Convolutional Layer

- There are three more hyperparameters for each CONV layer:
  - **Depth**: the number of neurons in the Conv layer that connect to the same region of the input volume. This is exactly the depth of the output volume from this layer. We will refer to a set of neurons that are all looking at the same region of the input as a **depth column**.
  - **Stride**: stride specifies how many pixels the receptive field moves each time, and it affects the spatial dimensions (width and height) of the output volume. Higher strides cause the receptive fields to overlap less and the resulting output volume to have smaller dimensions spatially.
  - The size of **zero-padding**: we can add zeros to the edges of the input to keep the output having the same size (width and height) as the input.

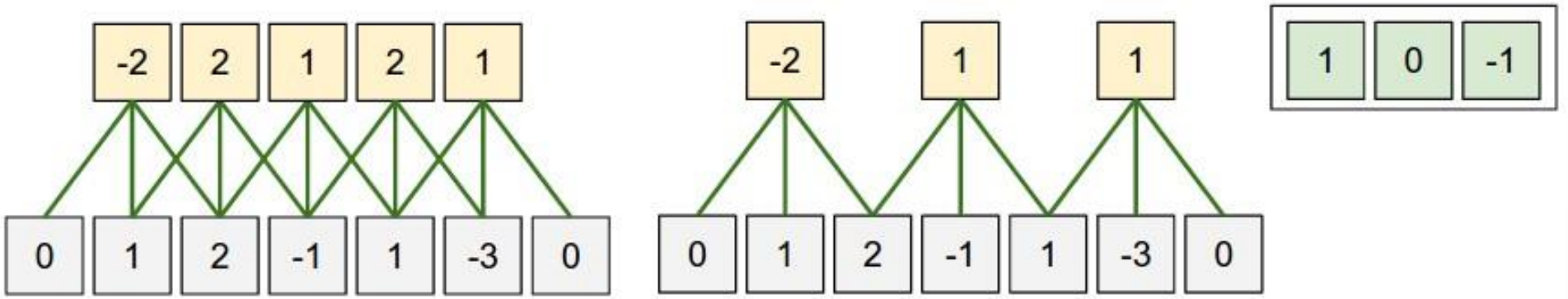
清华大学统计学研究中心

Convolutional Layer

- We can compute the spatial size of the output volume as a function of the input volume size (W), the receptive field size of the Conv Layer neurons (F), the stride with which they are applied (S), and the amount of zero padding used (P) on the border. The formula for calculating how many neurons "fit" is given by $(W - F + 2P)/S + 1$. This must be an integer.

清华大学统计学研究中心

Example



- In this example there is only one spatial dimension (x-axis), one neuron with a receptive field size of $F = 3$, the input size is $W = 5$, and there is zero padding of $P = 1$.
- Left: The neuron strode across the input in stride of $S = 1$, giving output of size $(5 - 3 + 2)/1 + 1 = 5$.
- Right: The neuron uses stride of $S = 2$, giving output of size $(5 - 3 + 2)/2 + 1 = 3$. Notice that stride $S = 3$ could not be used since it wouldn't fit neatly across the volume.

清华大学统计学研究中心

What is a Convolution?

- Mathematically, a convolution of two functions f and g is defined as

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau)\, \mathrm{d}\tau\,.$$

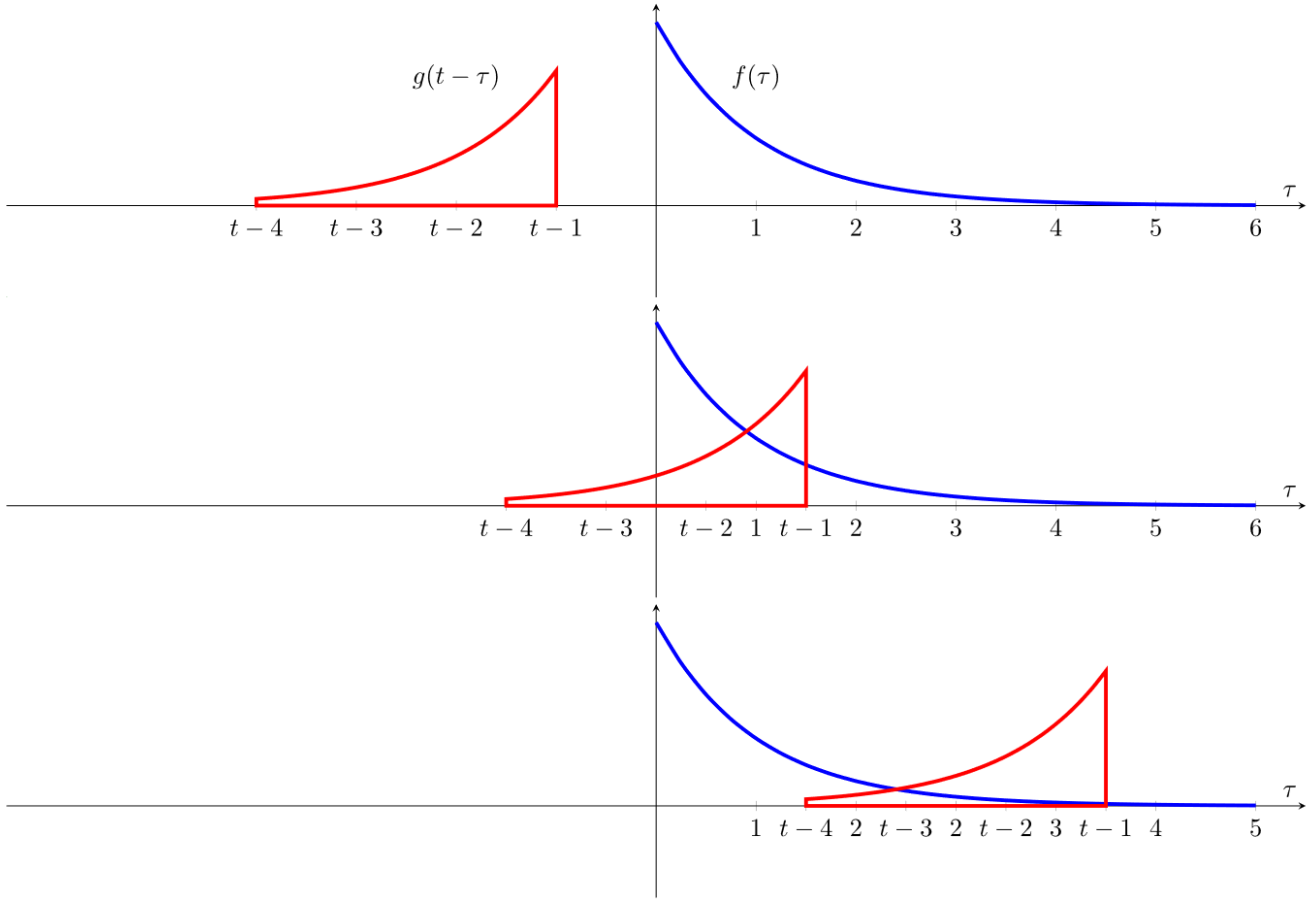
For discrete convolutions

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m] g[n - m]\,.$$

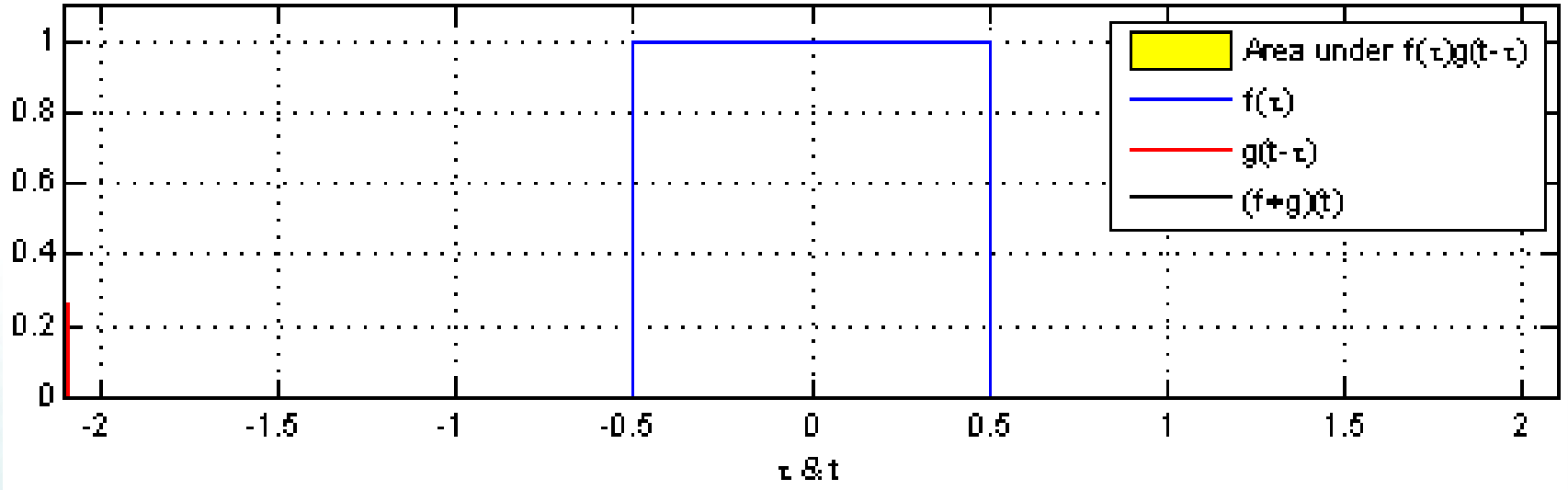
清华大学统计学研究中心

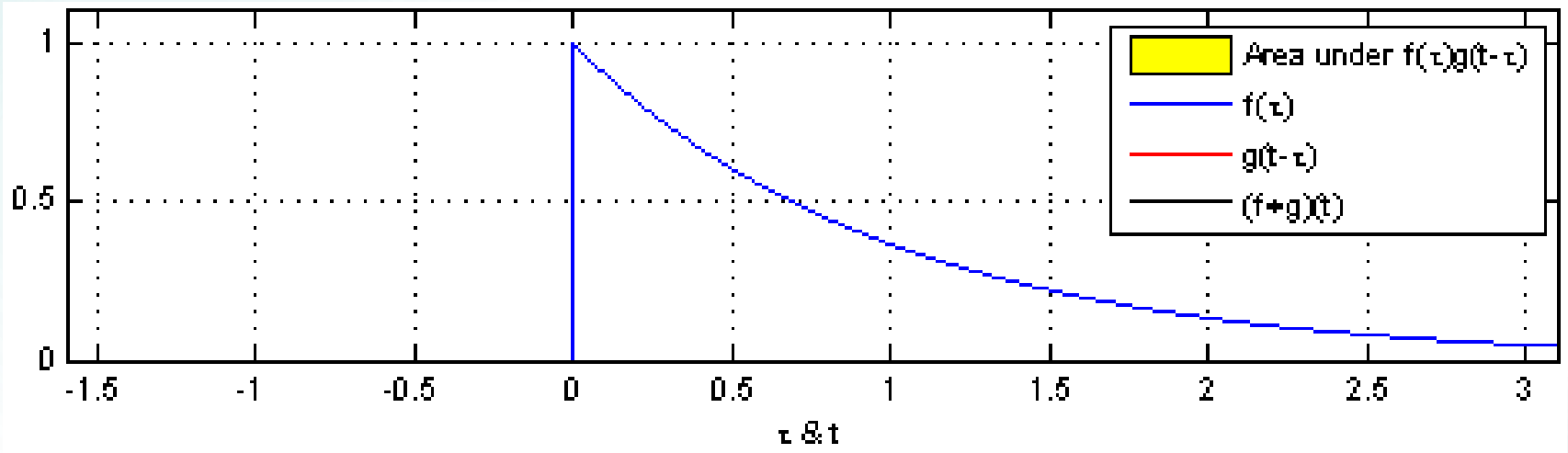
What is a Convolution?

- Practically, it can be understood as compute a sliding, weighted-sum of $f(\tau)$, where the weighting function is $g(-\tau)$.



What is a Convolution?





What is a Convolution?

- Convolutions are commonly used in image/audio processing, where they are called **filters** or **kernels**.

清华大学统计学研究中心

What is a Convolution?

- For example, we can blur an image by averaging pixels with surrounding ones.

0	0	0	0	0
0	1/9	1/9	1/9	0
0	1/9	1/9	1/9	0
0	1/9	1/9	1/9	0
0	0	0	0	0



What is a Convolution?

- Or find edges by looking for sharp changes in neighboring pixels.

0	0	0	0	0
0	0	0	0	0
0	-1	1	0	0
0	0	0	0	0
0	0	0	0	0



What is a Convolution?

- A practical interpretation of convolution for ConvNets: local filters using shared weights, applied to the entire input.

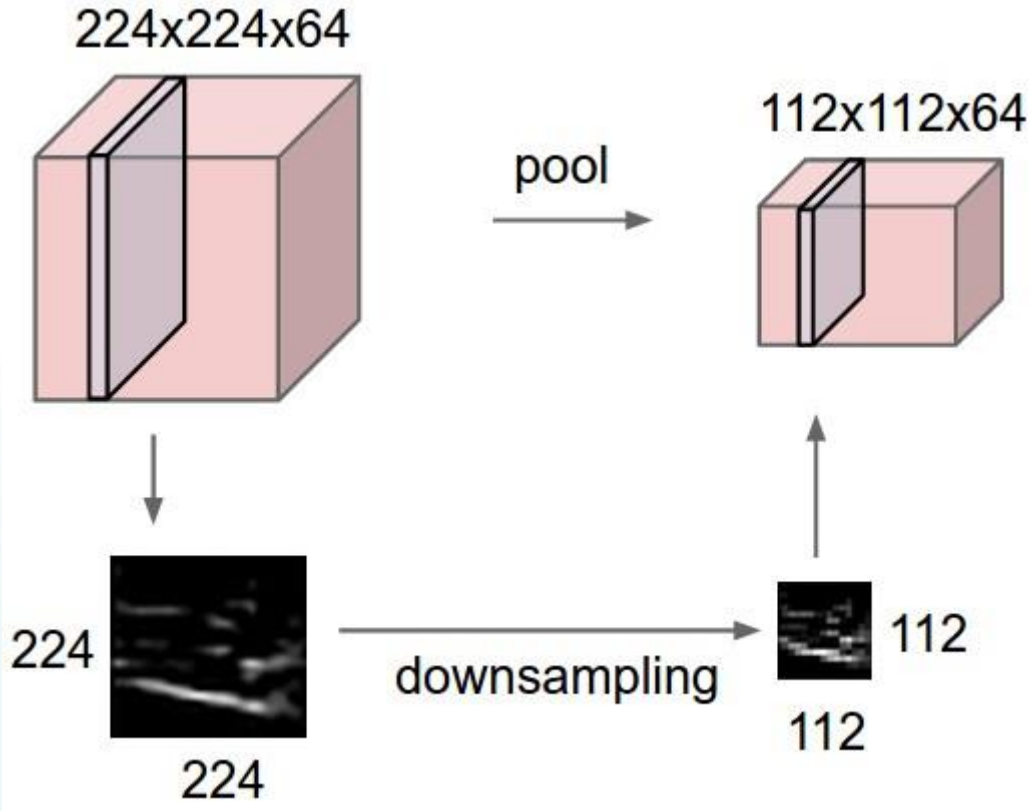
清华大学统计学研究中心

Pooling Layer

- It is common to periodically insert a Pooling layer in-between successive Conv layers in a ConvNet architecture. Its function is to progressively **reduce the spatial size** of the representation to reduce the amount of parameters and computation in the network, and hence to also control overfitting.
- The Pooling Layer operates independently on every depth slice of the input and resizes it spatially, using the MAX operation.
- The most common form is a pooling layer with filters of size 2×2 applied with a stride of 2 downsamples every depth slice in the input by 2 along both width and height, discarding 75% of the activations. Every MAX operation would in this case be taking a max over 4 numbers (little 2×2 region in some depth slice). The depth dimension remains unchanged.

清华大学统计学研究中心

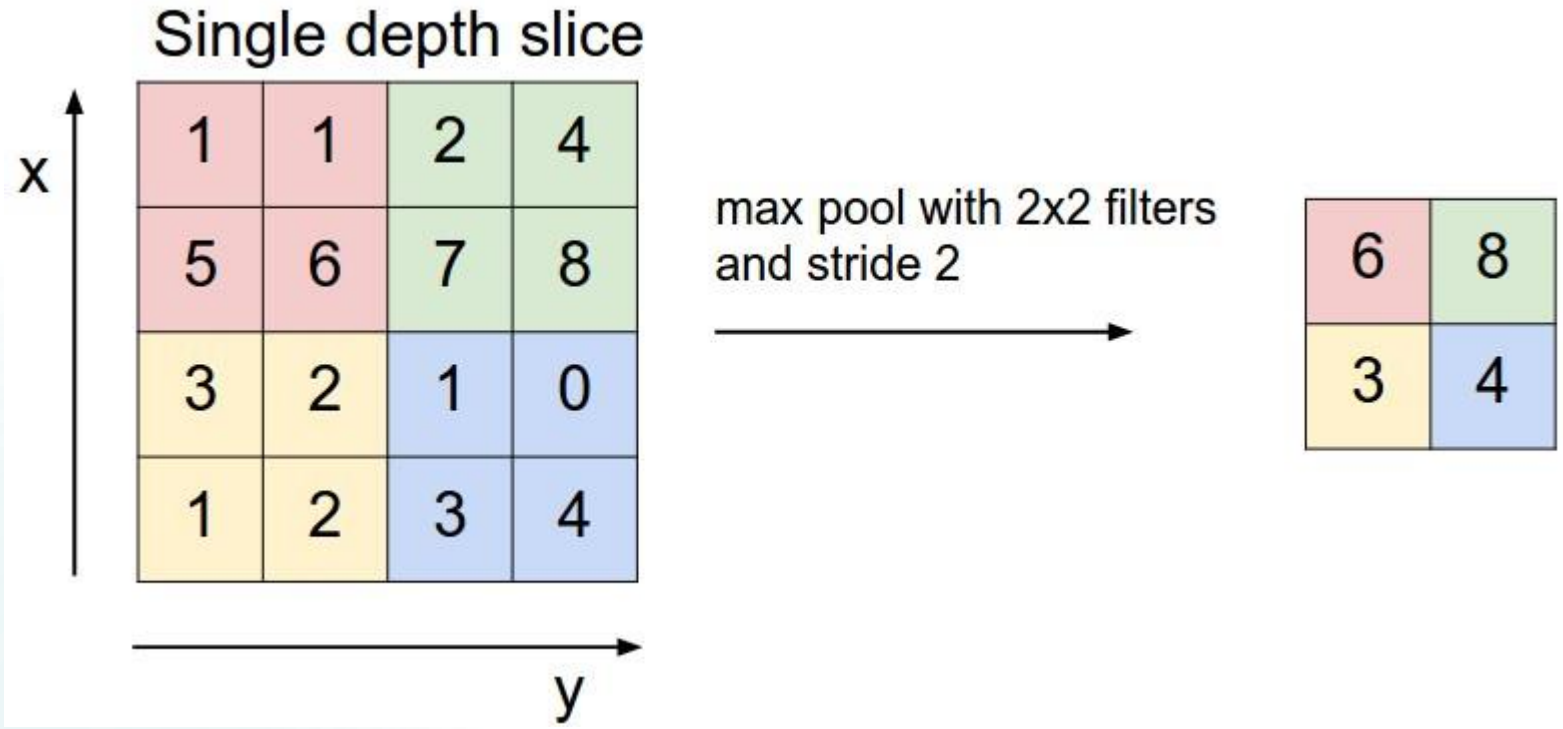
Pooling Layer



- Pooling layer downsamples the volume spatially, independently in each depth slice of the input volume. In this example, the input volume of size [224×224×64] is pooled with filter size 2, stride 2 into output volume of size [112×112×64]. Notice that the volume depth is preserved.

清华大学统计学研究中心

Pooling Layer



- The most common downsampling operation is max, giving rise to max pooling, here shown with a stride of 2. That is, each max is taken over 4 numbers (little 2×2 square).

清华大学统计学研究中心

Pooling Layer

- 聚合层

Verb [edit]

pool (*third-person singular simple present* **pools**, *present participle* **pooling**, *simple past and past participle* **pooled**)

1. (*transitive*) To put together; contribute to a common fund, on the basis of a mutual division of profits or losses; to make a common interest of. [quotations ▼]

   *We must **pool** our resources.*

2. (*intransitive*) To combine or contribute with others, as for a commercial, speculative, or gambling transaction.

Translations [edit]

±to put together	[show ▼]
±to combine or contribute with others	[show ▼]

https://en.wiktionary.org/wiki/pool

清华大学统计学研究中心

Fully-Connected Layer

- Note that an FC Layer can be seen as a CONV Layer as well.

- For example (AlexNet), an FC layer with $K = 4096$ that is looking at some input volume of size $7 \times 7 \times 512$ can be equivalently expressed as a CONV layer with $F = 7, P = 0, S = 1, K = 4096$.

- In other words, we are setting the filter size to be exactly the size of the input volume, and hence the output will simply be $1 \times 1 \times 4096$ since only a single depth column "fits" across the input volume, giving identical result as the initial FC layer.

清华大学统计学研究中心

ConvNet Architectures

- The most common ConvNet architecture follows the pattern:

```
INPUT -> [[CONV -> RELU]*N -> POOL?]*M -> [FC -> RELU]*K -> FC
```

- For example,

```
INPUT -> FC #implements a linear classifier.
INPUT -> CONV -> RELU -> FC
INPUT -> [CONV -> RELU -> POOL]*2 -> FC -> RELU -> FC
#a single CONV layer between every POOL layer.
INPUT -> [CONV -> RELU -> CONV -> RELU -> POOL]*3 -> [FC -> RELU]*2 -> FC
#Here we see two CONV layers stacked before every POOL layer.
#This is generally a good idea for larger and deeper networks,
#because multiple stacked CONV layers can develop more complex
#features of the input volume before the destructive pooling operation.
```

清华大学统计学研究中心

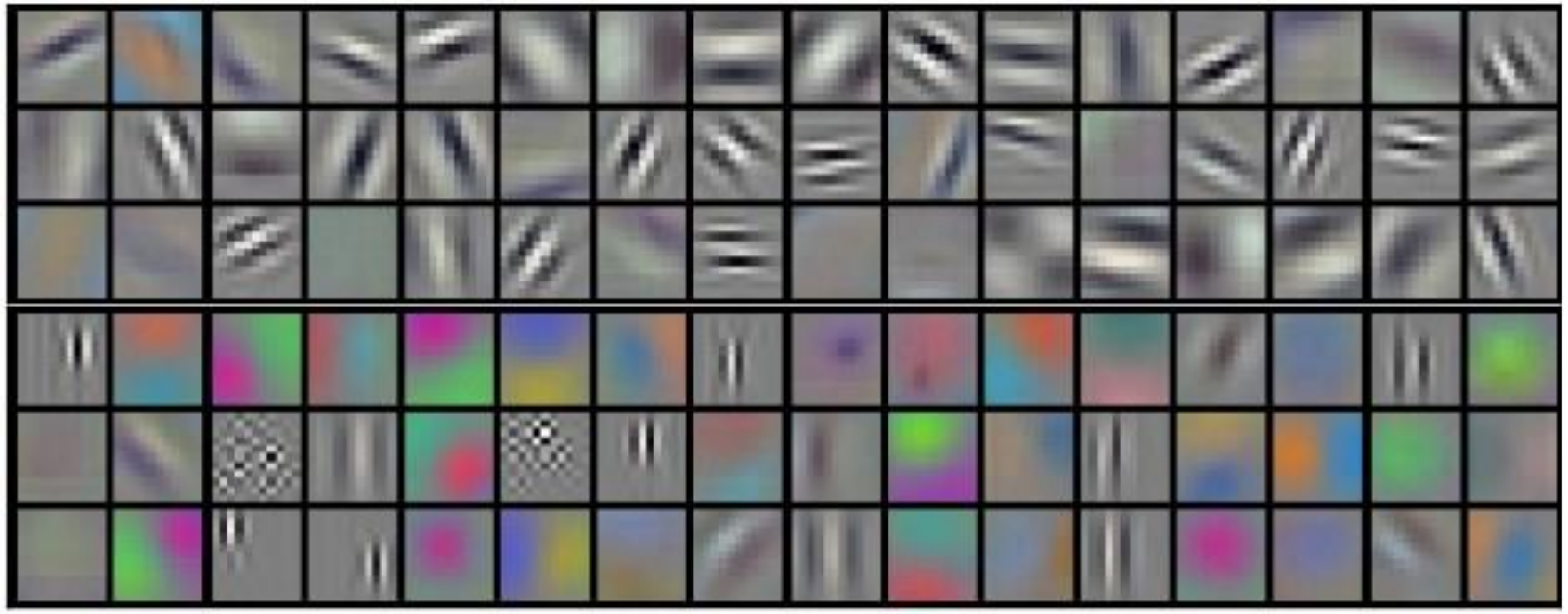
Visualizing ConvNets

- One common strategy is to visualize the weights. These are usually most interpretable on the first CONV layer which is looking directly at the raw pixel data.

- The weights are useful to visualize because well-trained networks usually display nice and smooth filters without any noisy patterns. Noisy patterns can be an indicator of a network that hasn't been trained for long enough, or possibly a very low regularization strength that may have led to overfitting.

清华大学统计学研究中心

Visualizing ConvNets



- Example filters learned by Krizhevsky et al. Each of the 96 filters shown here is of size [11×11×3], and each one is shared by the 55×55 neurons in one depth slice.

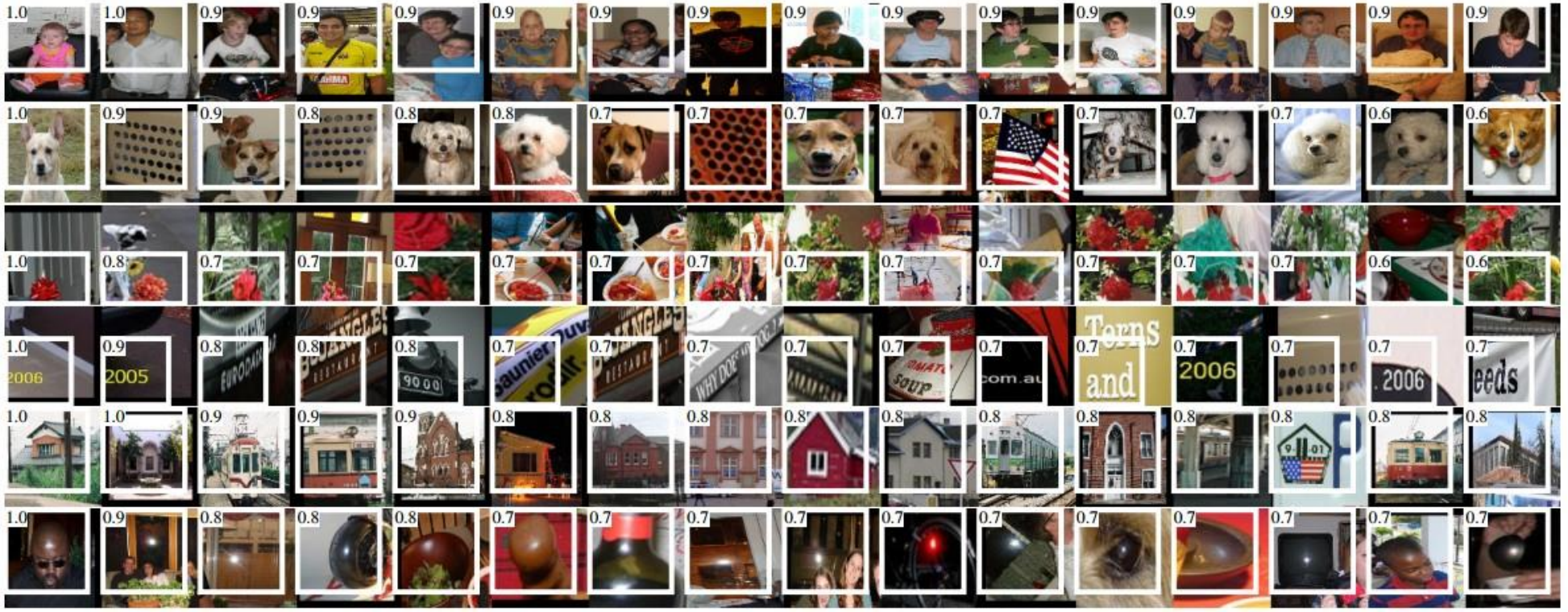
Visualizing ConvNets

- Another visualization technique is to take a large dataset of images, feed them through the network and keep track of which images maximally activate some neuron.

- We can then visualize the images to get an understanding of what the neuron is looking for in its receptive field.

清华大学统计学研究中心

Visualizing ConvNets



- Maximally activating images for some POOL5 (5th pool layer) neurons of an AlexNet. The activation values and the receptive field of the particular neuron are shown in white. (In particular, note that the POOL5 neurons are a function of a relatively large portion of the input image!)

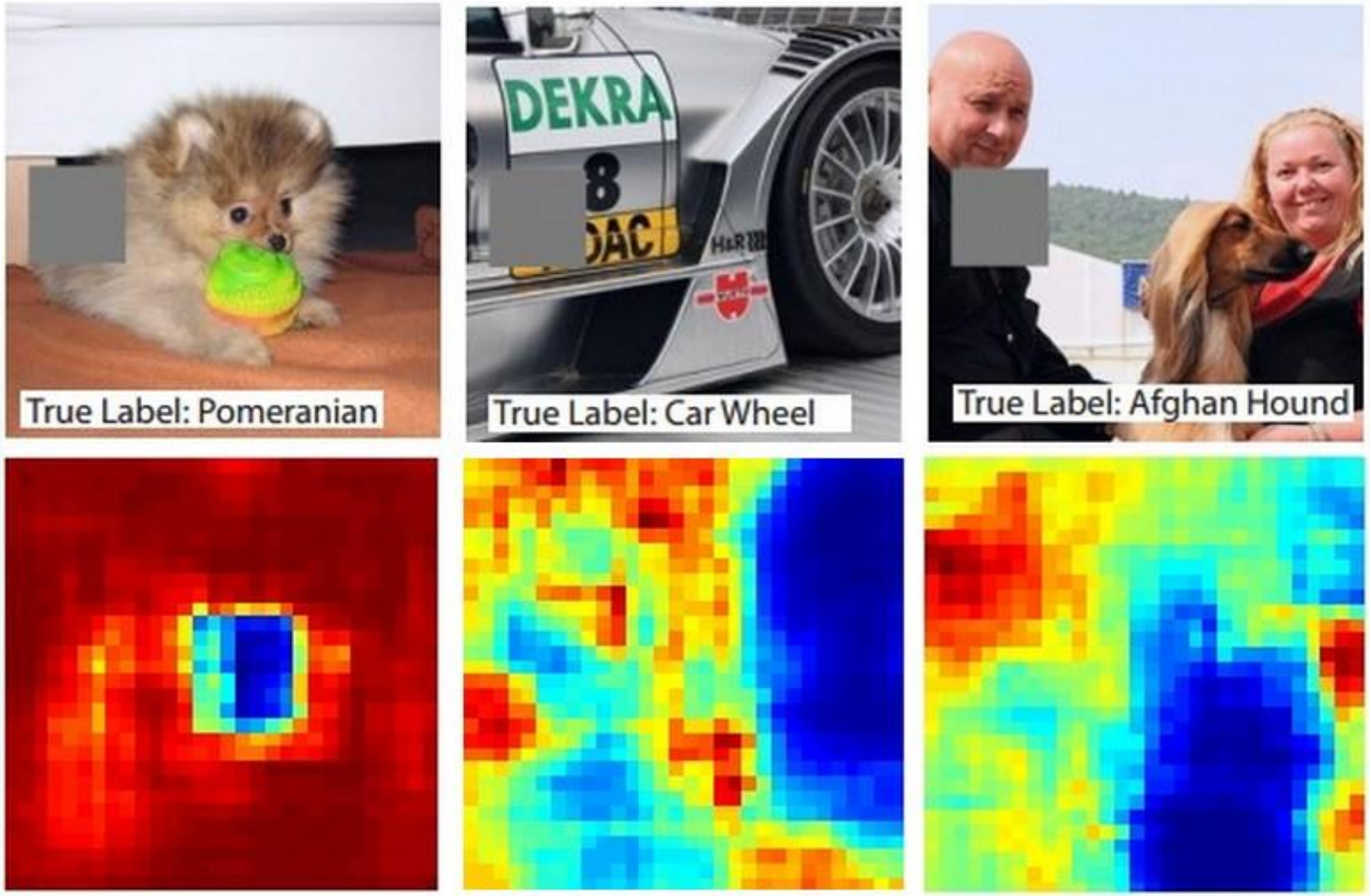
Visualizing ConvNets

- Suppose that a ConvNet classifies an image as a dog. How can we be certain that it's actually picking up on the dog in the image as opposed to some contextual cues from the background or some other miscellaneous object?

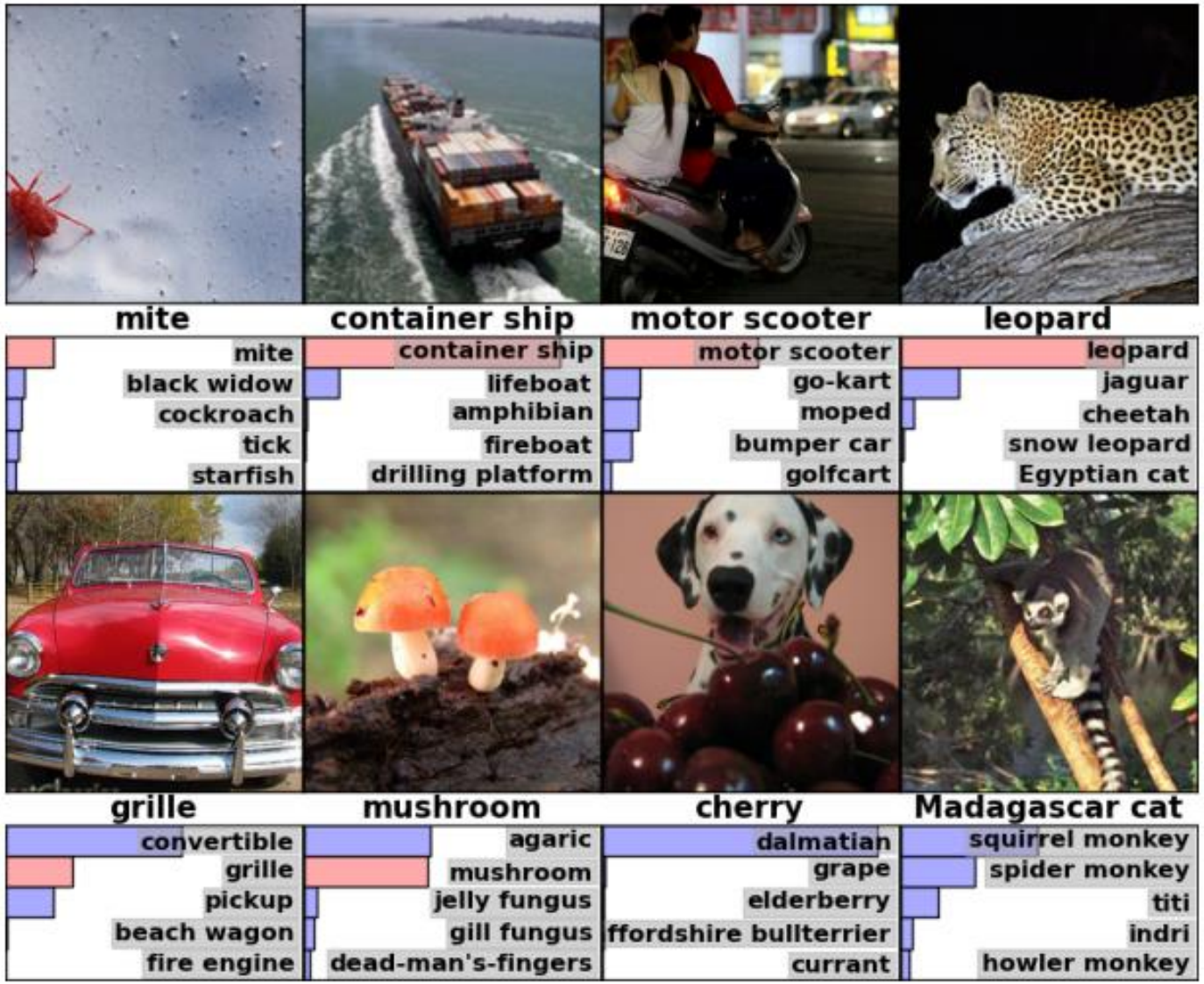
- One way of investigating which part of the image some classification prediction is coming from is by plotting the probability of the class of interest (e.g. dog class) as a function of the position of an occluded object. That is, we iterate over regions of the image, set a patch of the image to be all zero, and look at the probability of the class. We can visualize the probability as a 2-dimensional heat map.

清华大学统计学研究中心

Visualizing ConvNets

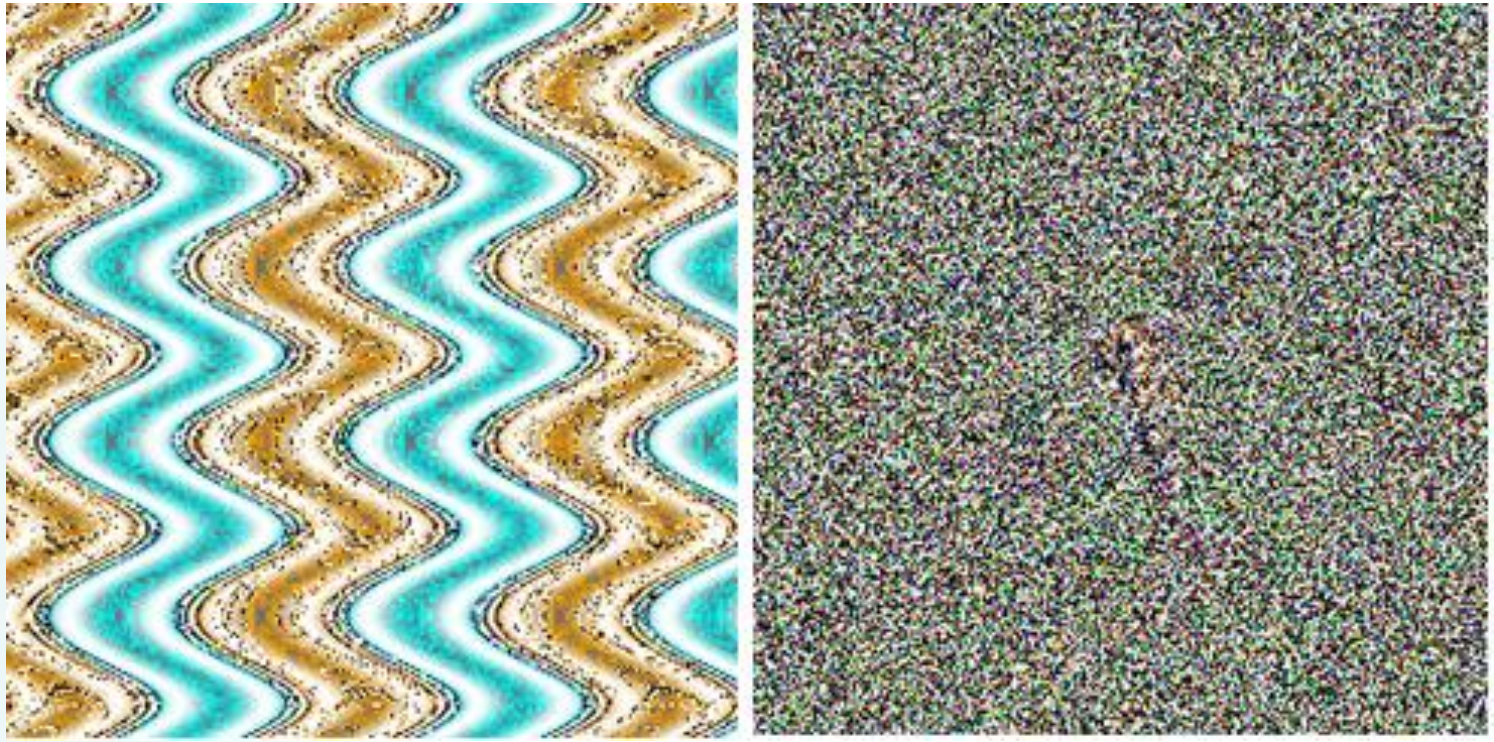


Performance



Performance

- Sometimes not so good.



Left: Starfish. Right: Cheetah.

Performance

- Sometimes not so good.



Left: Dog. Right: Ostrich.

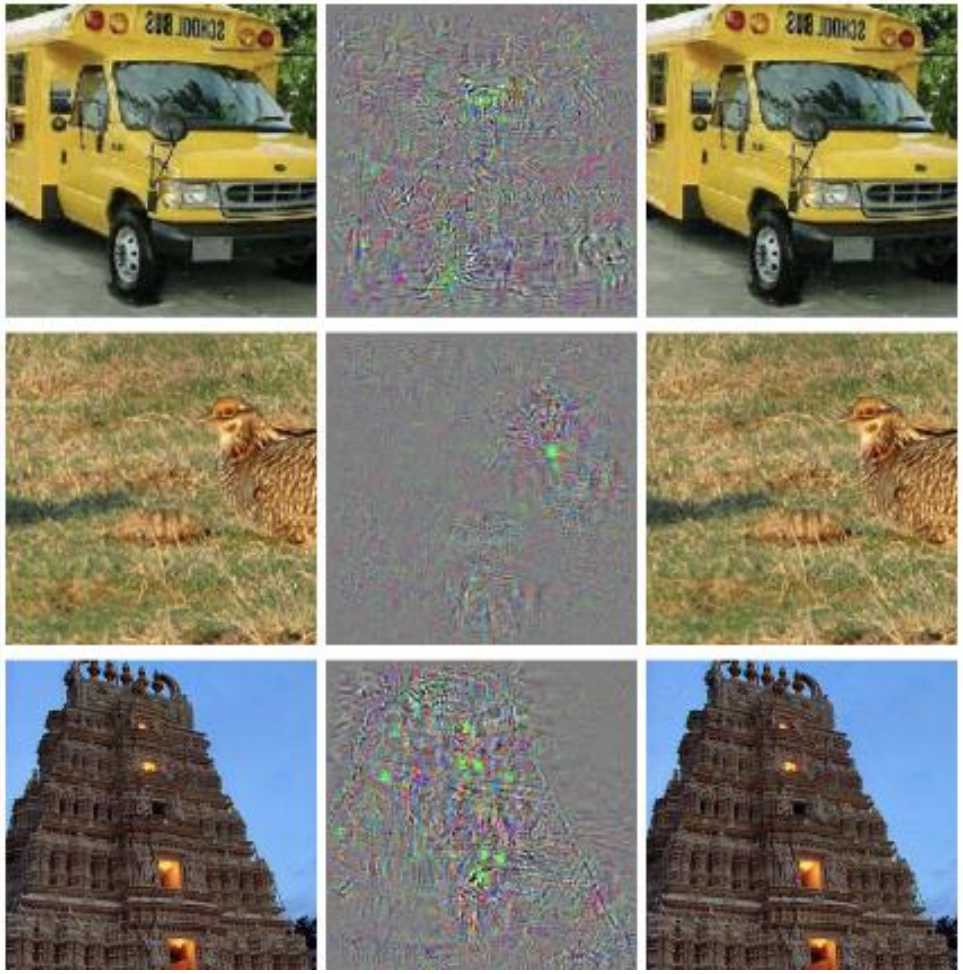
Adversarial Examples

- The previous example is known as **adversarial example** or **adversarial attack** – the image was modified by an imperceivable amount, and the result would be totally wrong.
- We would expect that well-trained models to be tolerant to small perturbations. However, all existing deep learning models (i.e., not only CNN) are vulnerable to adversarial attacks.
- Experiments have confirmed that adding true “white noise” to an image typically doesn’t impact the predictions of well-performing models. Adversarial examples are typically engineered.

清华大学统计学研究中心

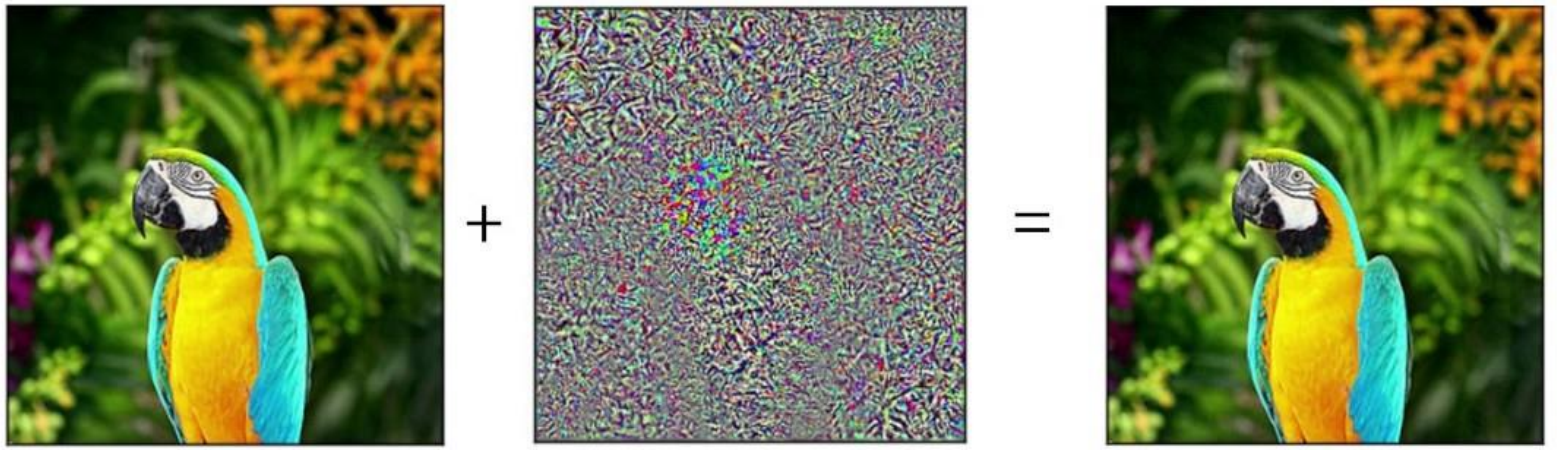
More Examples

- In the right-hand column we have: entirely giraffes.



Adversarial Attacks

- The earliest, and simplest technique, is called Fast Gradient Sign Method (Goodfellow et al., 2015).



- Other attack methods typically limit the L0/L1/L2 distance from the original image.

Recurrent Neural Networks

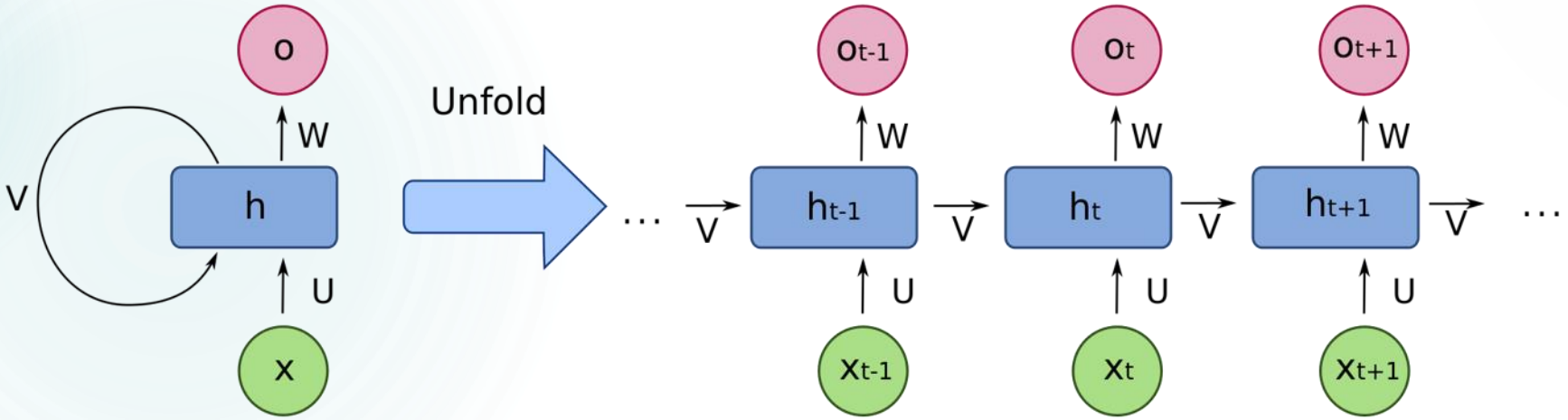
Motivation from Sentence Generation

- Question: how can you use neural networks to generate sentences?
- Key issues:
  - Should you use an entirely different set of weights to predict each character/word in the sentence?
  - How to make the generated character/word coherent with the preceding text?
  - The number of words/characters in a sentence is indefinite. How to determine the number of words/characters in the sentence?

清华大学统计学研究中心

A Basic Model

- The basic **Recurrent Neural Network (RNN)** model looks like below, where x_t is the input character/word of the tth step, o_t is the output of the tth step that predicts the next character/word. The h_t is the hidden variable of the network, which represents the cumulated information of the sentence, and is passed through the characters/words.



清华大学统计学研究中心

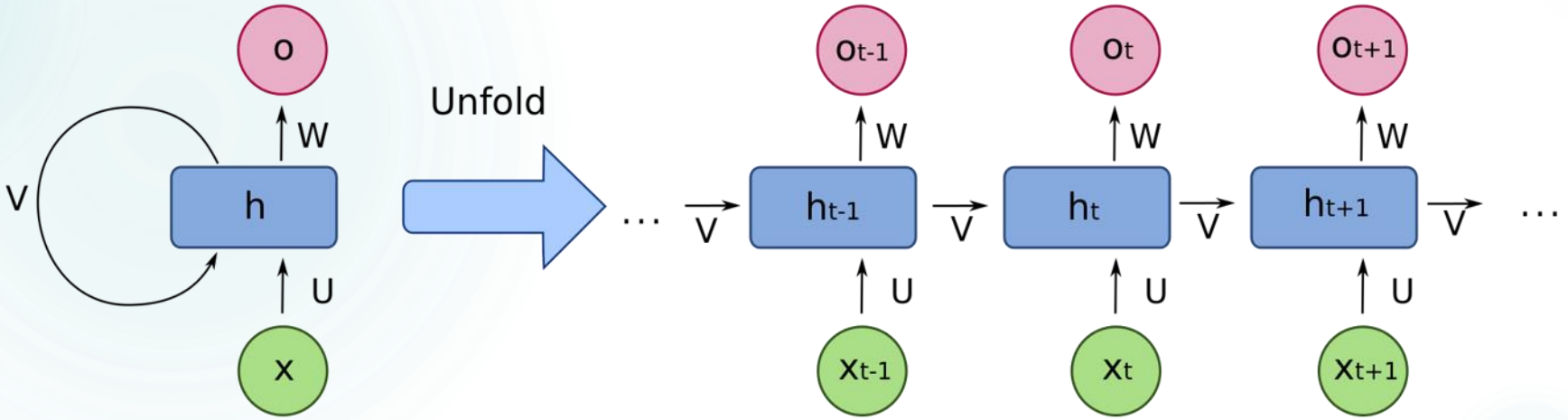
A Basic Model

- As illustrated, h_t is obtained from the previous cumulated information h_{t-1} and the new input x_t:

$$h_t = f(Ux_t + Vh_{t-1} + b_h).$$

- The output o_t is obtained from h_t. Since o_t is discrete, use softmax to normalize the probabilities:

$$o_t = softmax(Wh_t + b_o).$$



清华大学统计学研究中心

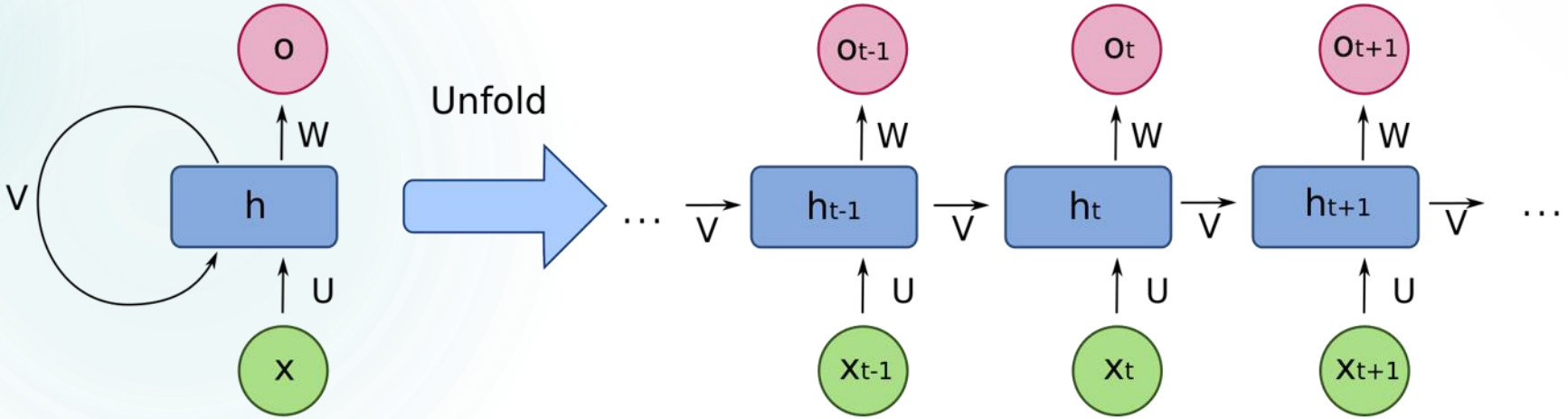
When does the sentence end?

- The RNN model can generate unlimited words and never stop.
- Use special tokens to indicate **Sentence_Start** and **Sentence_End**.
  - Use **Sentence_Start** to be x_0.
  - When the prediction if **Sentence_End**, stop the sentence generation.

清华大学统计学研究中心

Training RNN

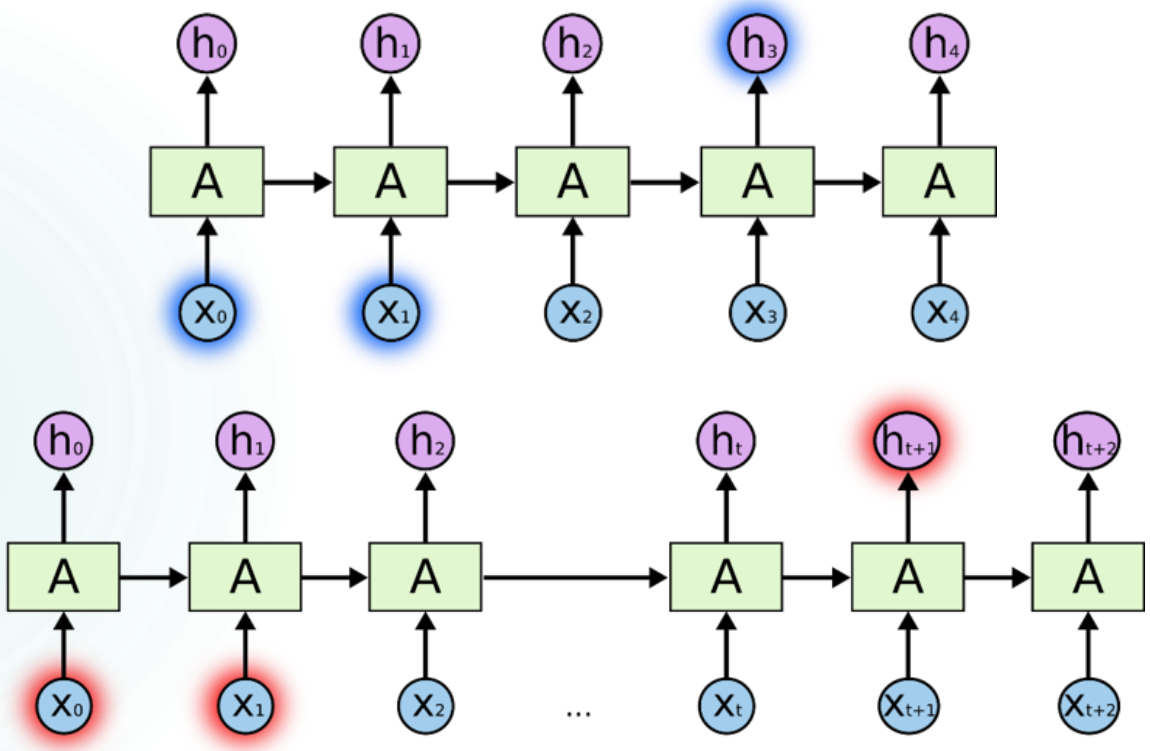
- For the sentence generation problem, o_t is the prediction of x_{t+1}, and x_{t+1} is the true label, which makes it a classification problem, and cross-entropy is usually used as the loss function.
- The loss at step t will be back-propagated to all the preceding steps to update U, V, W, h_h and b_o.



清华大学统计学研究中心

Limitations of the Basic Model

- Researchers have found that it is hard to train the basic RNN model to learning long distance dependency - i.e., the prediction of the next token is only affected by the previous few (say, 2) tokens.



清华大学统计学研究中心

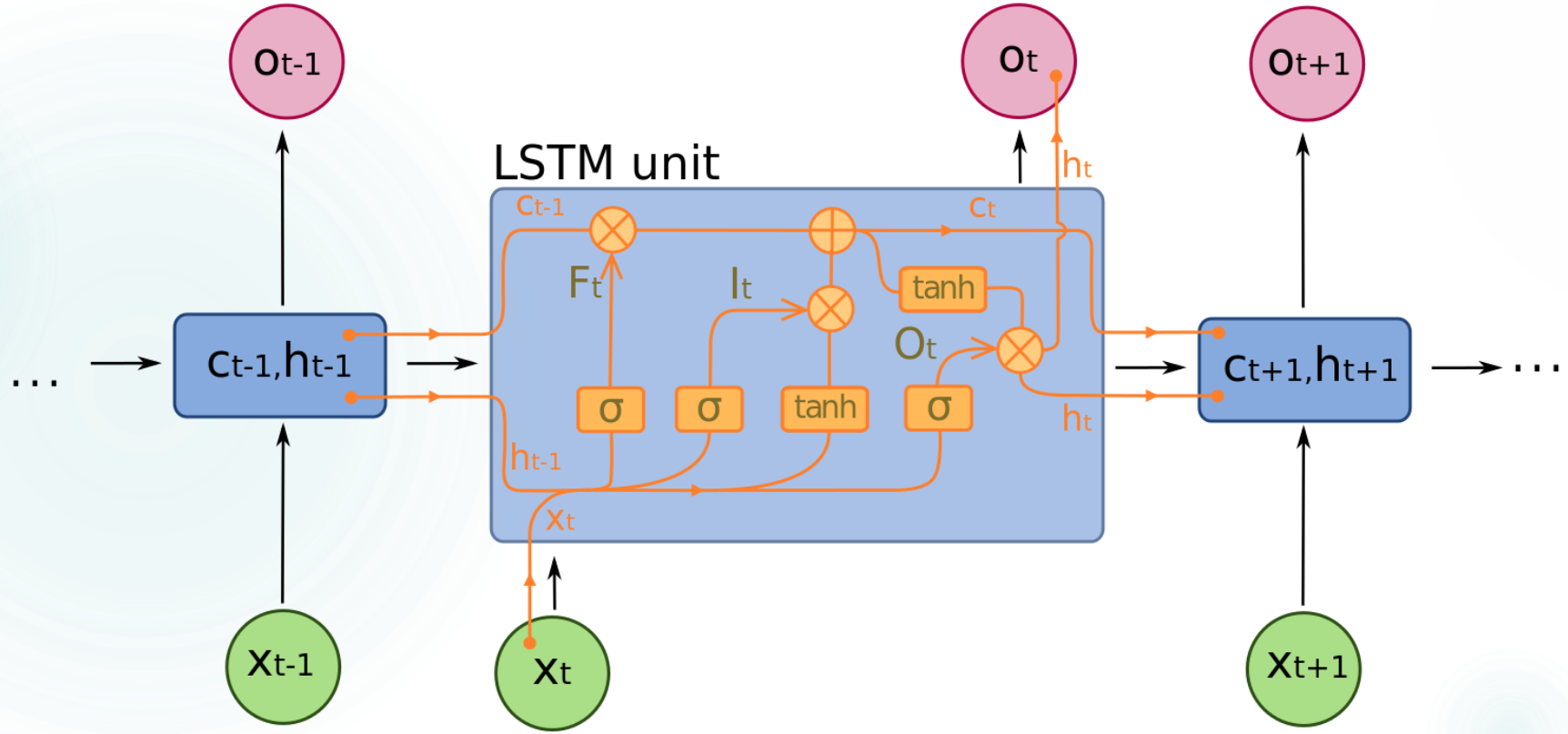
Limitations of the Basic Model

- Why does this happen?
  - Try to calculate $\partial y/\partial a$ where $y = f\left(a\left(f\left(af(ax)\right)\right)\right)$, a is a scalar, and f is the logistic function.
  - The **vanishing gradient problem**!
  - The basic RNN is not competent for language problems.

清华大学统计学研究中心

LSTM

- The Long Short Term Memory (LSTM) model was proposed in 1997 by Sepp Hochreiter & Jürgen Schmidhuber. It became famous since winning a competition in 2003.

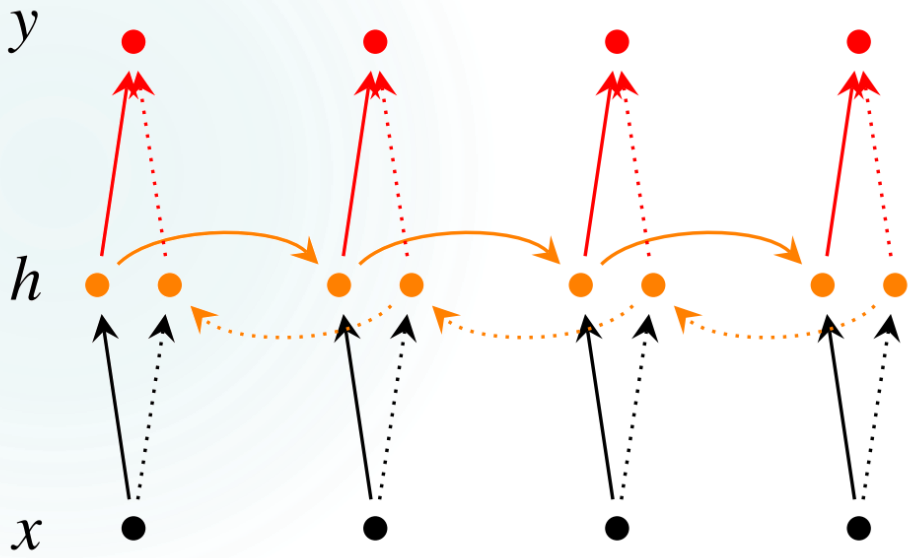


LSTM

- Forget gate: $f_t = \sigma\left(W_f[h_{t-1}, x_t] + b_f\right)$
- Input gate: $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$
- Candidate content: $\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$
- Cell state: $C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$
- Output gate: $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$
- Output: $h_t = o_t \cdot \tanh(C_t)$

Various uses of RNN

- The application of RNN is not limited to sequence prediction.
- One can also take the whole sequence as input, and only make a single prediction at the last step. In this way, the RNN is used to summarize the information of the input sequence.
- Even with models such as the LSTM, RNNs still put more emphasis on the last few tokens. If you suspect that all the tokens are important, use bi-directional RNN (LSTM or GRU).



$$\vec{h}_t = f(\vec{W}x_t + \vec{V}\vec{h}_{t-1} + \vec{b})$$
$$\overleftarrow{h}_t = f(\overleftarrow{W}x_t + \overleftarrow{V}\overleftarrow{h}_{t+1} + \overleftarrow{b})$$
$$y_t = g(U[\vec{h}_t; \overleftarrow{h}_t] + c)$$

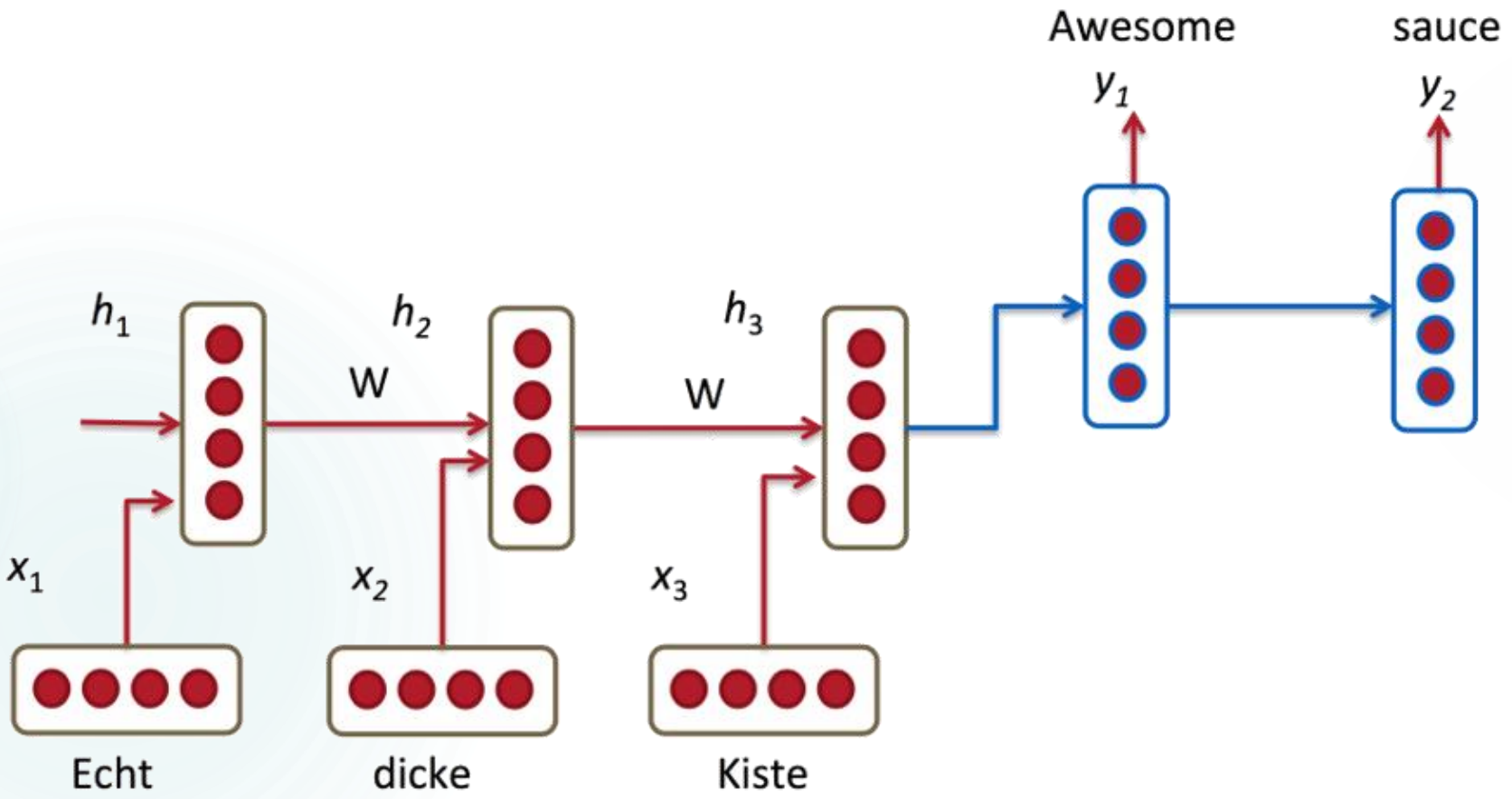
清华大学统计学研究中心

Various uses of RNN

- One impressive capability of RNNs is that the hidden vector (h_t in the basic RNN, C_t or h_t in LSTM) can selectively compress information from the sentence to a numeric vector!
- The most notable application of this capability is machine translation using the "**Encoder-Decoder**" model. (proposed by Cho et al. 2014)

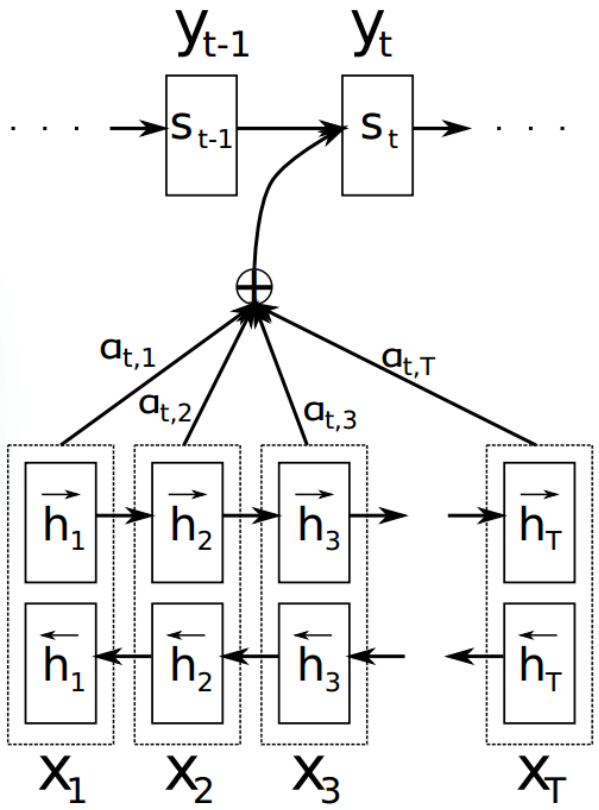
清华大学统计学研究中心

Encoder-Decoder



Encoder-Decoder

- Cho et al.'s model was only good at translating short phrases, but bad at long text. It was not good at learning long distance dependencies.
- In the same year, Bahdanau et al. proposed an model with attention mechanism to solve this issue.



清华大学统计学研究中心

Attention mechanism

$$P(y_i|y_1,\dots,y_{i-1},x) = g(y_{i-1},s_i,c_i)$$

$$s_i = f(s_{i-1},y_{i-1},c_i)$$

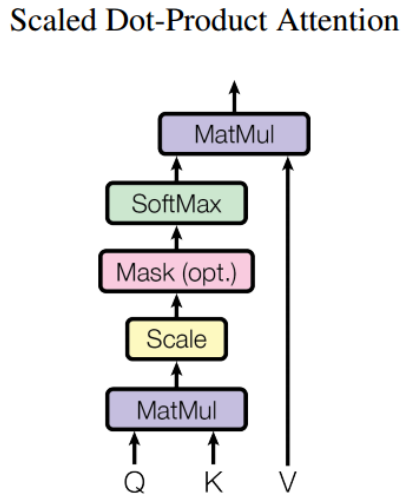
$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

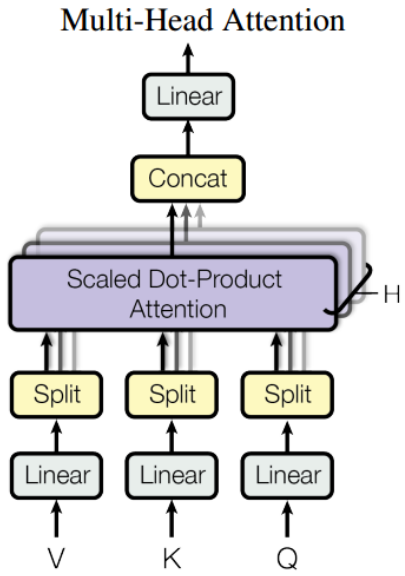
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$$

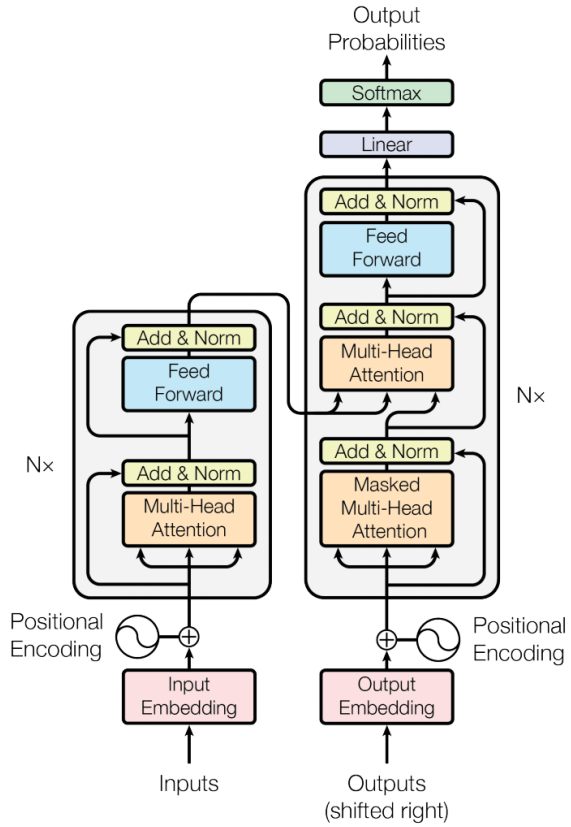
$$e_{ij} = a\left(s_{i-1},h_j\right)$$

Transformer

- In 2017, Google proposed the Transformer model, which still follows the Encoder-Decoder idea but bypasses the RNN mechanism for speed.
- Transformer has been the backbone model architecture for language ever since.







Milestones to Large Language Models



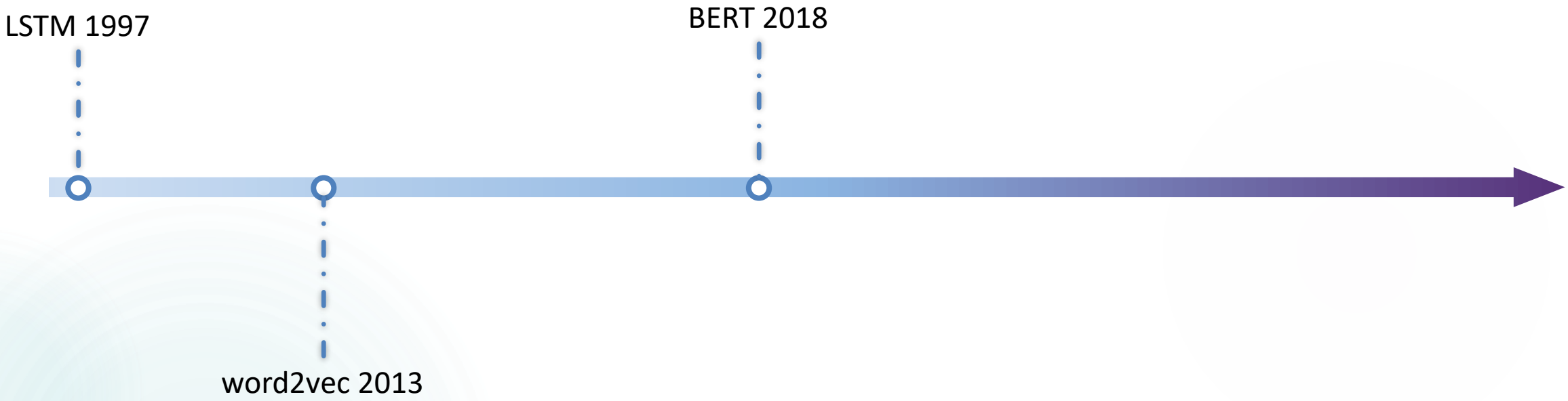
- LSTM: early day model architecture for language
  - Starting to be popular since 2007
  - Limited capability

Milestones to Large Language Models



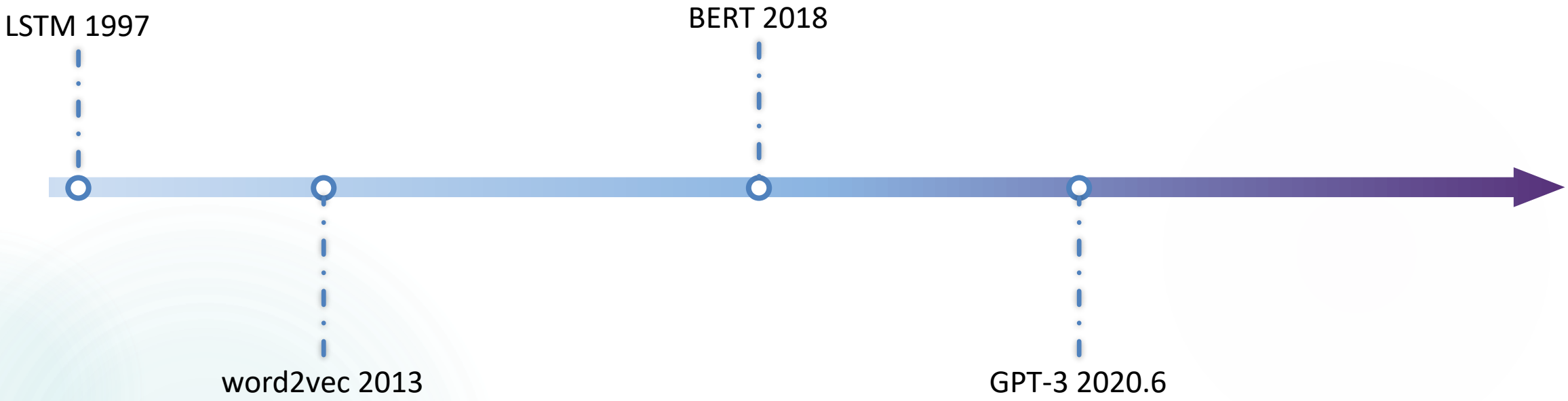
- Word2vec: semantic representation of words using vectors
  - The magic of unsupervised pre-training is beginning to sink in
  - Not a language model

Milestones to Large Language Models



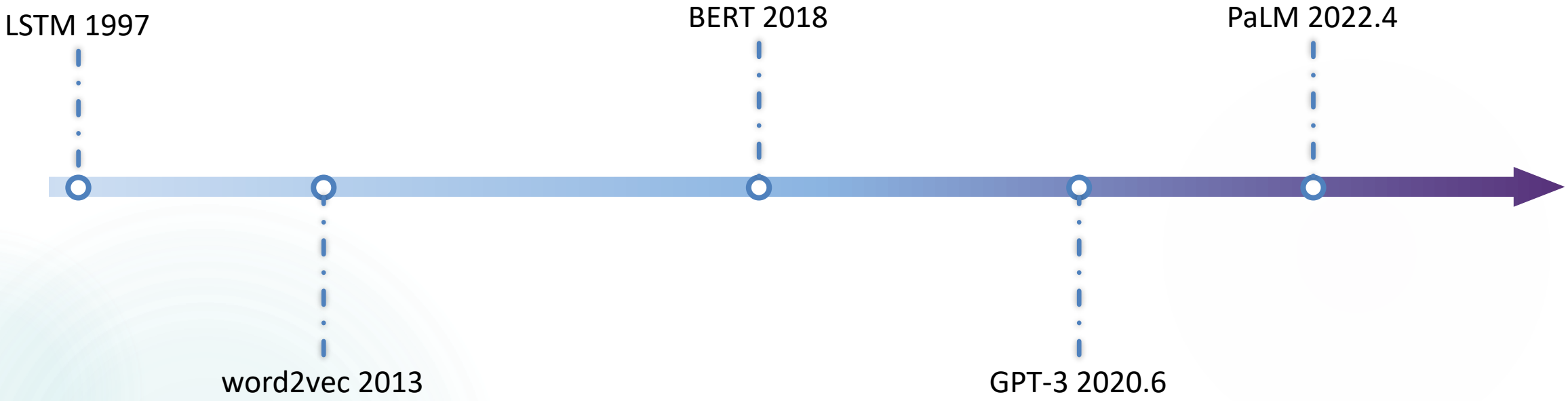
- BERT (340 million parameters)
  - A pretrained Transformer encoder that significantly outperforms almost all previous models in all tasks.
  - Introducing the new paradigm of “pretraining + fine-tuning”.

Milestones to Large Language Models



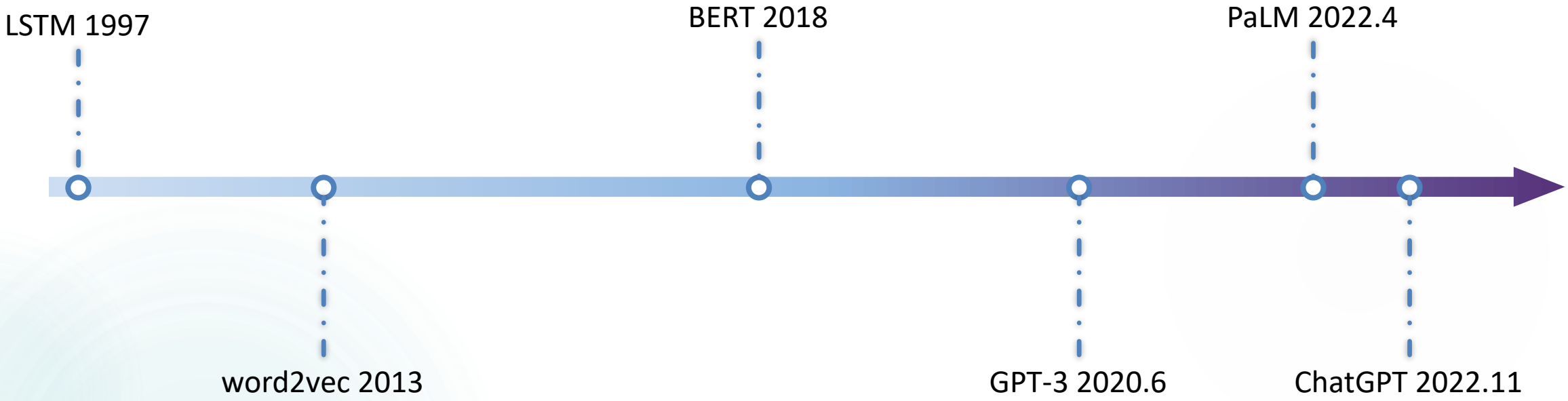
- GPT-3 (175B parameters)
  - A stochastic parrot that seems to show intelligence?
  - Introduces the concept of “in-context learning”.

Milestones to Large Language Models



- PaLM (540B parameters)
  - Introduces the concept of “chain-of-thought”
  - Significant multi-step reasoning capability.

Milestones to Large Language Models



- ChatGPT (est. 10~30B)
  - Alignment to human values/goals; Instruction following; Multi-turn conversation;
  - Substantial world knowledge; human-like reasoning and reading comprehension;
  - The starting point of a new industrial revolution.